MODULE 07

MongoDB Programming in Python

Documents, replica sets and sharding, PyMongo CRUD, queries, aggregation and indexing



```
>>> from pymongo import MongoClient
```

AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering

Learning Objectives

By the end of this module, you will be able to:

1. Explain MongoDB's document model and map relational concepts to collections, documents and fields
2. Describe how replica sets and sharded clusters provide availability and horizontal scaling
3. Run MongoDB 8 with Docker, a local install or Atlas, and use `mongosh` and Compass
4. Connect with `MongoClient` and perform CRUD operations with PyMongo 4 result objects
5. Design document schemas: embedding vs referencing, BSON types and `$jsonSchema` validation
6. Query with filters, projections, sorting and pagination, and build aggregation pipelines
7. Speed up queries with indexes and verify them with `explain()`
8. Access MongoDB asynchronously with PyMongo's `AsyncMongoClient`

MODULE 07 · MONGODB PROGRAMMING

Module Roadmap

01 **Introduction to MongoDB**
Documents, collections and how they compare with relational tables

02 **MongoDB Architecture**
From a single server to replica sets and sharded clusters

03 **Installation and Tools**
MongoDB 8 in Docker, on your machine or in the cloud — plus `mongosh` and Compass

04 **PyMongo Basics**
Connecting from Python: clients, databases, collections and cursors

05 **CRUD Operations**
Create, read, update and delete — in mongosh and in PyMongo

06 **Data Types and Schema Design**
BSON types in mongosh and Python, and enforcing structure with $jsonSchema

07 **ObjectIds and Relations**
Identity, creation time and linking documents

08 **Querying Data**
Filters, projections, operators, sorting and pagination

09 **Aggregation Pipelines**
Server-side data processing, stage by stage

10 **Indexing**
From collection scans to index scans

11 **Async PyMongo**
AsyncMongoClient with asyncio and FastAPI

01

Introduction to MongoDB

Documents, collections and how they compare with relational tables

- What MongoDB is and when it fits
- Databases, collections, documents and `_id`
- Relational vs MongoDB terminology
- Rules for keys: types, case, duplicates and order

What Is MongoDB?

Document database

Stores **documents**: JSON-like records with nested objects and arrays

Flexible schema

Documents in one collection may differ; add fields without a migration

BSON storage

Documents are stored and transferred as typed binary JSON

Scales out

Replica sets for availability, sharding for horizontal scaling

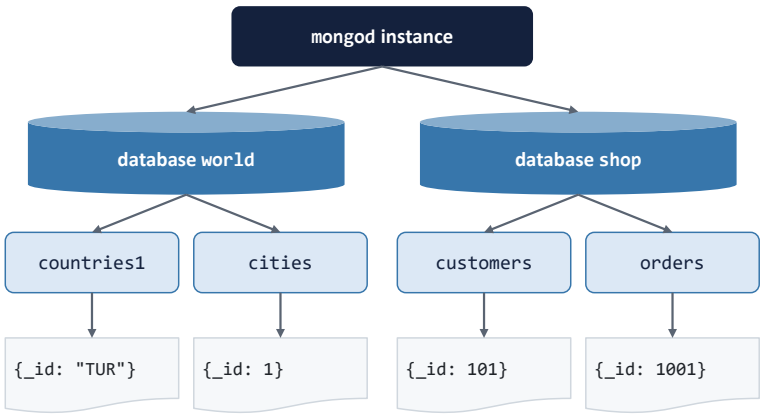
one order document

```
{
  "_id": 1001,
  "customer": {
    "name": "Ada Lovelace",
    "city": "London"
  },
  "items": [
    {"product": "Notebook", "qty": 2},
    {"product": "Pen", "qty": 10}
  ],
  "status": "shipped"
}
```

Editions

MongoDB 8.x Community Server is free to use; MongoDB Atlas is the managed cloud service.

Databases, Collections and Documents



- One server hosts many independent **databases**
- A database groups **collections** — tables with a dynamic schema
- A collection holds **documents** — rows, but far more expressive
- Every document has a unique `_id` field within its collection

Relational Databases vs MongoDB

Relational (SQL)	MongoDB	Difference
Database	Database	Same idea: a namespace with its own permissions
Table	Collection	No fixed columns
Row	Document	May nest objects and arrays
Column	Field	Can differ from document to document
Primary key	`_id` field	Always present, unique and indexed
Foreign key + `JOIN`	Embedding or reference (+ `$lookup`)	Data read together is stored together
Schema (`CREATE TABLE`)	Flexible; optional `$jsonSchema`	Validation only if you add it
SQL	MQL: `find()`, `aggregate()`	Queries are documents, not strings

Note

MongoDB supports multi-document ACID transactions (since 4.0), but a single-document write is already atomic — good document design rarely needs transactions.

Documents Are Ordered Key–Value Pairs

A **document** is an ordered set of **keys** (field names) with associated **values** — in Python it maps naturally to a `dict`.

mongosh

```
// one key
{ greeting: "Hello, world!" }

// several keys
{ greeting: "Hello, world!", views: 3 }
```

```
>>> page = {"greeting": "Hello, world!", "views": 3}
>>> page["views"] += 1
>>> page
{'greeting': 'Hello, world!', 'views': 4}
```

Values

Values can be strings, numbers, booleans, `null`, dates, arrays and nested documents — the BSON types of Section 6.

Rules for Field Names

- Field names (keys) are **strings**; any UTF-8 character is allowed
- Never use the null character (`U+0000`): BSON uses it to end a key
- A leading `$` is reserved for **operators**: `$set`, `$gt`, `$push`
- A `.` means **dot notation**: `"address.city"` reaches into nested documents
- `_id` is reserved for the primary key

PyMongo

```
customers.update_one(
    {"address.city": "Ankara"},  # dot notation: a nested field
    {"$set": {"tier": "gold"}},  # $-prefixed: an update operator
)
```

Watch out

MongoDB 5.0+ can store names that contain `$` or `.`, but many operators cannot address them — avoid them in your own schemas.

Types, Case and Duplicate Keys

mongosh: three distinct documents

```
{ views: 3 }      // a number
{ views: "3" }    // a string: another value
{ Views: 3 }      // another key
```

```
>>> page = {"greeting": "Hello, world!",
...         "greeting": "Hello, MongoDB!"}
>>> page
{'greeting': 'Hello, MongoDB!'}
```

Key idea

MongoDB is **type-sensitive** and **case-sensitive**: a filter `{views: 3}` matches only the first document.

Watch out

A document cannot contain the same key twice. A Python `dict` literal with a repeated key silently keeps the **last** value.

Field Order in Documents

```
>>> import bson
>>> first = {"x": 1, "y": 2}
>>> second = {"y": 2, "x": 1}
>>> first == second
True
>>> bson.encode(first) == bson.encode(second)
False
```

Python compares dicts by content; BSON keeps the order of the keys.

Watch out

Matching a **whole embedded document** is order-sensitive: `{size: {h: 14, w: 21}}` does not match `{size: {w: 21, h: 14}}`.

Tip

Use dot notation instead: `{"size.h": 14, "size.w": 21}` — and never design a schema that depends on field order.

QUIZ

Quick Check: Documents

Which statement about MongoDB documents is true?

A. A document may repeat a key if the two values differ

B. `{"views": 3}` and `{"views": "3"}` store the same value

C. Every document in a collection has a unique `_id`

D. All documents in a collection must have the same fields

ANSWER

Quick Check: Documents

Which statement about MongoDB documents is true?

- A. A document may repeat a key if the two values differ
- B. `{"views": 3}` and `{"views": "3"}` store the same value
- C. **Every document in a collection has a unique `_id`**
- D. All documents in a collection must have the same fields

Why

`_id` is the primary key: present in every document and unique within its collection. Keys cannot repeat, `3` and `"3"` are different types, and collections have a flexible schema.

TRY IT

Model a Relational Record as a Document

A relational database stores students and their enrolments in two tables. Write the equivalent **single** MongoDB document as a Python `dict`.

id	name	email
2021001	Elif Kaya	elif@example.edu

table `students`

student_id	course	grade
2021001	AIN3005	A
2021001	AIN3001	B+

table `enrollments`

Tip

Hint: the student number becomes `_id`; the enrolments become an array of sub-documents.

ANSWER

Model a Relational Record as a Document

```
student = {
    "_id": 2021001,
    "name": "Elif Kaya",
    "email": "elif@example.edu",
    "enrollments": [
        {"course": "AIN3005", "grade": "A"},
        {"course": "AIN3001", "grade": "B+"},
    ],
}
print(student["enrollments"][0]["course"])
```

OUTPUT

```
AIN3005
```

- The primary key `id` becomes `_id`
- The foreign key `student_id` disappears: enrolments live inside their owner
- One read returns the whole record — no `JOIN`

02

MongoDB Architecture

From a single server to replica sets and sharded clusters

- Components: drivers, `mongod`, `mongos`, config servers
- Replica sets: primary, secondaries, elections
- Sharding: shard keys and query routing
- Schema design: embedding vs referencing

2 · MONGODB ARCHITECTURE

Architecture at a Glance



Standalone

One `mongod` process

- development and tests
- no redundancy

Replica set

Several `mongod` copies of the same data

- one primary, several secondaries
- high availability

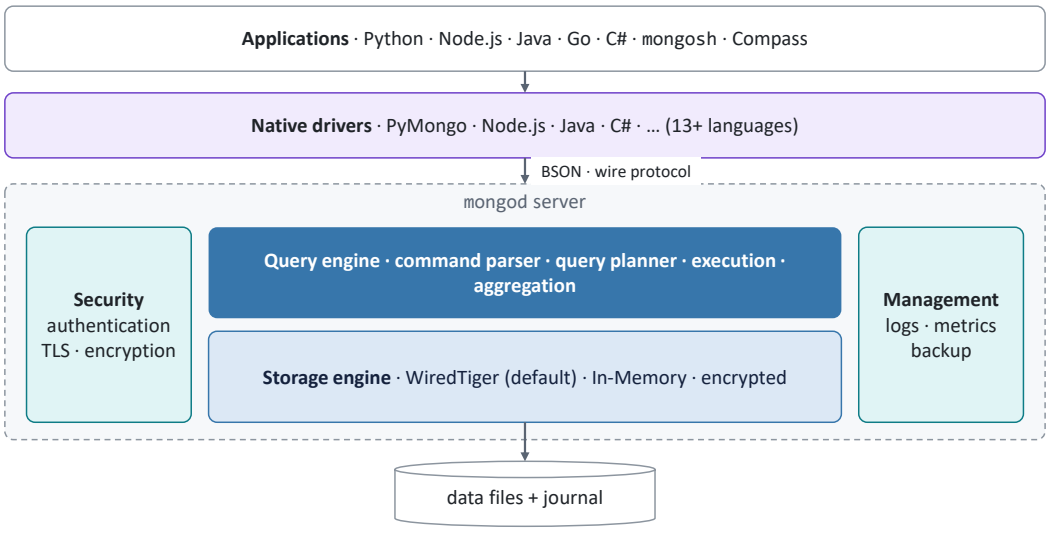
Sharded cluster

Data split across **shards**

- `mongos` routers + config servers
- horizontal scaling

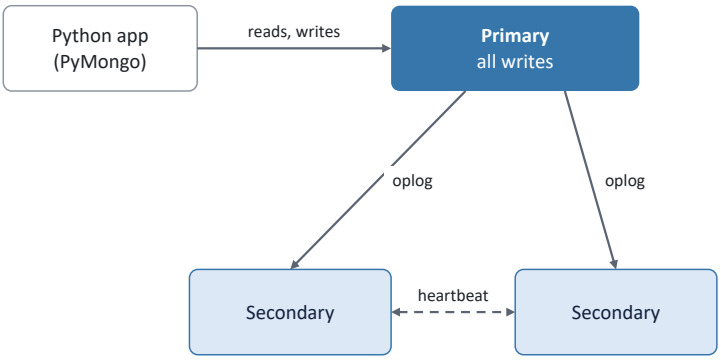
2 · MONGODB ARCHITECTURE

Inside a mongod Server



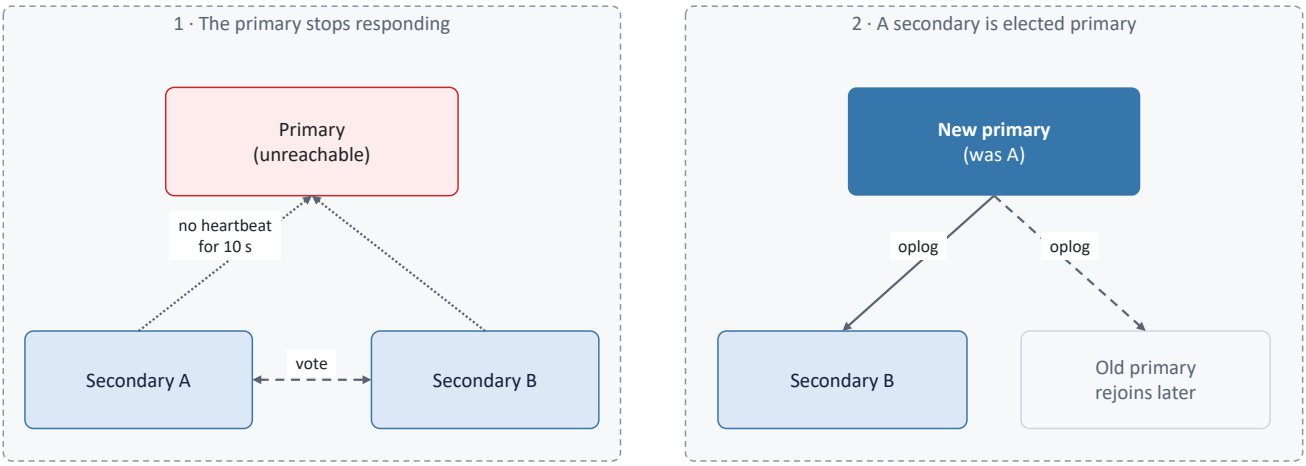
Redrawn after the original figure. The old MMAPv1 storage engine was removed in MongoDB 4.2.

Replica Sets: Copies for High Availability



- A **replica set** is a group of `mongod` processes holding the same data
- The **primary** accepts all writes and records them in the **oplog**
- **Secondaries** copy the oplog and apply it asynchronously
- Members exchange **heartbeats** every 2 seconds
- Typical size: 3 members, an odd number for elections

Automatic Failover and Elections



- An election needs votes from a **majority** of the members
- Writes pause briefly (typically < 12 s); drivers **retry** a failed write once automatically

Reads, Writes and Read Preference

Read preference	Reads from
`primary` (default)	the primary only
`primaryPreferred`	primary; a secondary if none
`secondary`	secondaries only
`secondaryPreferred`	secondaries; primary if none
`nearest`	the member with the lowest latency

- **Writes** always go to the primary (through `mongos` in a cluster)
- **Write concern** `w`: how many members must confirm a write
- `w="majority"` is the default since MongoDB 5.0
- Reads from secondaries may be slightly **stale**

PyMongo

```
client = MongoClient(
    "mongodb://db1,db2,db3/?replicaSet=rs0",
    readPreference="secondaryPreferred",
    w="majority",
)
```


QUIZ

Quick Check: Write Concern

A replica set has three members and the application writes with `w="majority"`. When is the write acknowledged?

A. As soon as the primary has applied it

B. When the primary and at least one secondary have applied it

C. Only when all three members have applied it

D. After a secondary has been elected primary

ANSWER

Quick Check: Write Concern

A replica set has three members and the application writes with `w="majority"`. When is the write acknowledged?

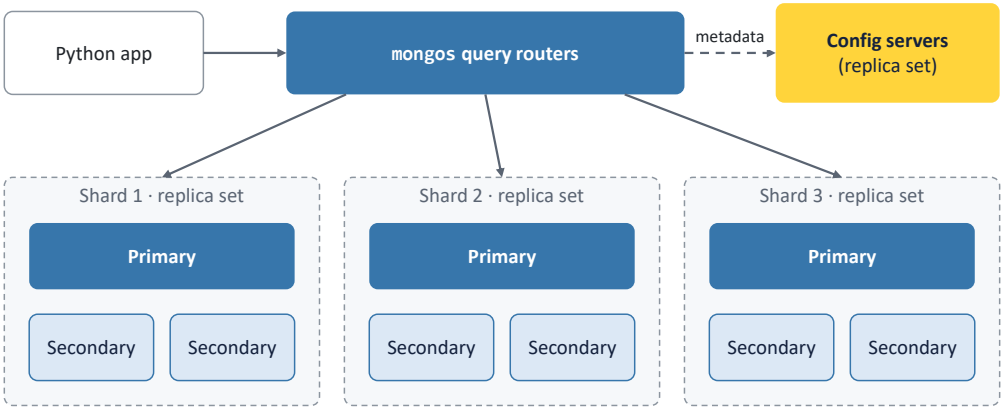
- A — As soon as the primary has applied it
- **B — When the primary and at least one secondary have applied it**
- C — Only when all three members have applied it
- D — After a secondary has been elected primary

Why

A majority of three is two members, and the primary counts. The write then survives the loss of any single member.

2 · MONGODB ARCHITECTURE

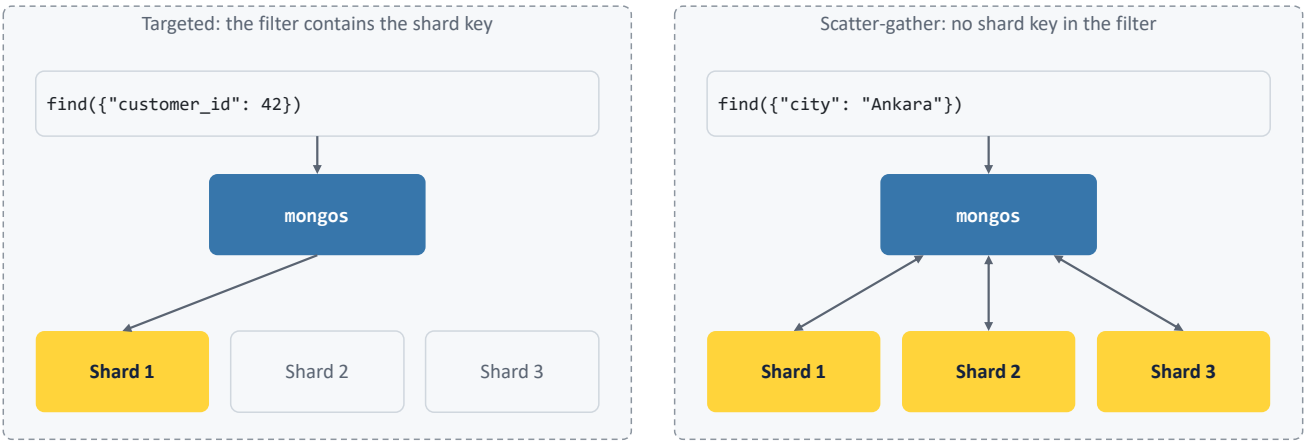
Sharding: Scaling Out a Cluster



Key idea

Shards each hold a subset of the data, **config servers** store which chunk lives where, and **mongos** routes every operation to the right shard(s).

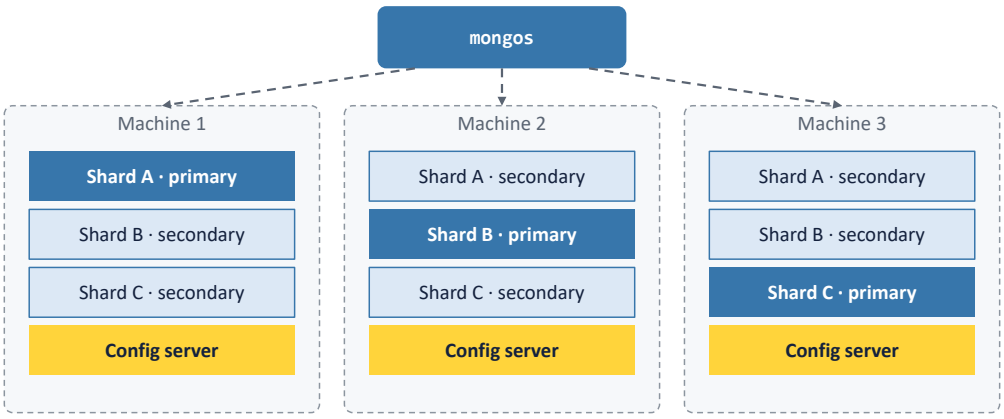
Routing Queries Through mongos



Key idea

Include the **shard key** in frequent queries so that `mongos` can target a single shard.

Microsharding: Several Shards per Machine



Note

Each machine runs several `mongod` processes. Primaries are spread out, so every machine takes a share of the writes and one failure costs each shard only one member.

2 · MONGODB ARCHITECTURE

Choosing a Shard Key

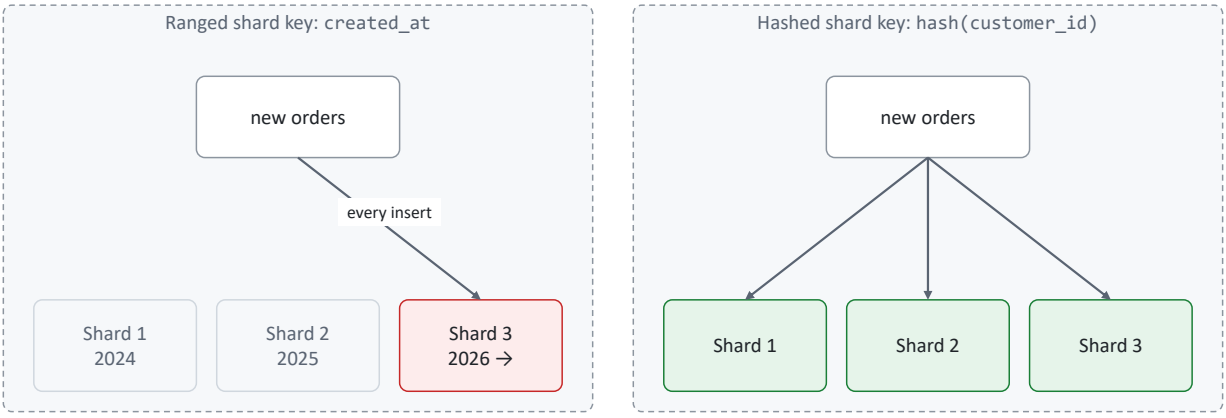
Criterion	Why it matters	Good	Poor
High **cardinality**	Many distinct values allow many chunks	`customer_id`, `email`	`country`, `status`
Even **frequency**	No single value dominates a chunk	user ids	`"guest"` in 60 % of documents
Not **monotonic**	Inserts spread over all shards	hashed `_id`	timestamps, `ObjectId`
Matches **queries**	Frequent queries become targeted	`product_id` if you query by product	a field no query uses

mongosh

```
sh.shardCollection("shop.orders", { customer_id: "hashed" })
```


2 · MONGODB ARCHITECTURE

Hot Spots: Ranged vs Hashed Shard Keys



Watch out

Monotonically increasing keys (timestamps, counters, `ObjectId`) send every insert to the last chunk: one **hot shard** does all the work. Hash the key or choose another one.

QUIZ

Quick Check: Shard Keys

An `orders` collection receives thousands of inserts per second and is queried mostly by customer. Which shard key is best?

A `{created_at: 1}` (ranged)

B `{status: 1}` — four possible values

C `{customer_id: "hashed"}`

D `{_id: 1}` with default `ObjectId` values

ANSWER

Quick Check: Shard Keys

An `orders` collection receives thousands of inserts per second and is queried mostly by customer. Which shard key is best?

A `{created_at: 1}` (ranged)

B `{status: 1}` — four possible values

C `{customer_id: "hashed"}`

D `{_id: 1}` with default `ObjectId` values

Why

A hashed `customer_id` has high cardinality, spreads inserts evenly and keeps the frequent per-customer queries targeted.

2 · MONGODB ARCHITECTURE

Embedding vs Referencing

Embedding: one `orders` document

```
{"_id": 1001,
 "customer": {"name": "Ada",
              "city": "London"},
 "items": [
   {"product": "Pen", "qty": 10},
   {"product": "Ink", "qty": 2}]}
```

Referencing: two collections

```
customers
{"_id": 42, "name": "Ada",
 "city": "London"}
```

`customer_id` → `_id`

```
orders
{"_id": 1001, "customer_id": 42,
 "items": [...]}
```

Embed when…

- data is read together
- the child belongs to one parent
- arrays stay bounded

Reference when…

- data is large or shared by many parents
- it changes independently
- arrays would grow without limit (16 MB)

2 · MONGODB ARCHITECTURE

Schema Design for Read and Write Workloads

Workload	Prefer	Avoid
Read-heavy	Embed data that is read together; **projections**; indexes on filter and sort fields	Several queries or `$lookup` per page view
Write-heavy	Small documents; few indexes; bulk writes; targeted `$set` / `$inc` updates	Duplicated data that must be updated in many places; unbounded arrays
Every workload	Model around the application's queries	Copying a normalised SQL schema one-to-one

✗ **Common misconception**

Denormalising (duplicating data) speeds up **reads**, not writes: every copy must be updated. Embedding helps writes only when one `update_one` can change everything atomically.

DISCUSS

Design a Blog Schema

A blog page shows one **post**, its **author's** name and all **comments**. A post gets up to a few hundred comments; an author writes many posts and may change their display name.

1. Embed or reference the comments? Why?
2. Embed or reference the author? What about the name shown on the page?
3. Posts are read by their id. Which shard key would you choose?

Format

Discuss in pairs for 4 minutes, then sketch one `posts` document.

ANSWER

Design a Blog Schema

- **Comments: embed** — read with the post, bounded, owned by one post
- **Author: reference** `author_id`, plus a copy of the `name` for display
- Rare name changes update the copies (`update_many`)
- **Shard key:** hashed `_id` — even spread, targeted reads by id

posts

```
{
  "_id": 501,
  "title": "Why BSON?",
  "author": {"_id": 42, "name": "Ada"},
  "comments": [
    {"by": "Linus", "text": "Nice!"},
    {"by": "Grace", "text": "+1"}
  ]
}
```

03

Installation and Tools

MongoDB 8 in Docker, on your machine or in the cloud — plus `mongosh` and Compass

- Four ways to run MongoDB
- Docker, a local install and MongoDB Atlas
- The `mongosh` shell and its basic commands
- Command-line tools and MongoDB Compass

Ways to Run MongoDB

Docker
Official `mongo:8` image; one command, easy to reset
Recommended for this course

Community Server 8.x
Free native install: Windows MSI, macOS Homebrew, Linux `apt` / `dnf`

MongoDB Atlas
Managed cloud service with a free **M0** cluster (512 MB)

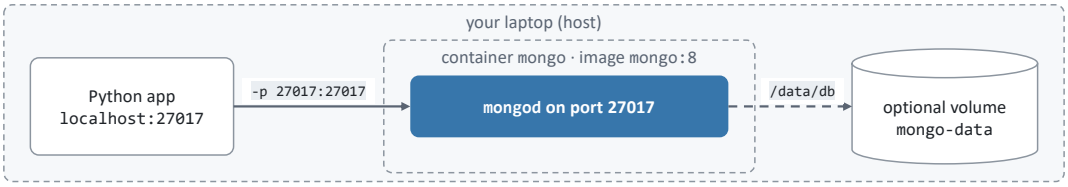
Enterprise and Ops Manager
Commercial edition and on-premises management — not needed here

Note
The tools are separate downloads: `mongosh`, Compass and the **Database Tools** (`mongodump`, `mongoimport`, …).

Running MongoDB with Docker

TERMINAL

```
$ docker run -d --name mongo -p 27017:27017 mongo:8
$ docker exec -it mongo mongosh --quiet
test> db.runCommand({ ping: 1 })
{ ok: 1 }
test> exit
$ docker stop mongo      # later: docker start mongo
```



Tip

Keep the data in a named volume so it survives `docker rm`: add `-v mongo-data:/data/db`.

Watch out

The container has **no authentication**. On shared networks publish only to localhost:

`-p 127.0.0.1:27017:27017`

Installing MongoDB Locally

- Install **MongoDB Community Server 8.x** (MSI, Homebrew or `apt`/`dnf`)
- Server binaries: `mongod` (database) and `mongos` (router)
- Data directory: default `/data/db` (Windows: `C:\data\db`)
- Default port: **27017**
- Install `mongosh` and Compass separately

TERMINAL

```
$ mkdir -p ~/mongo-data
$ mongod --dbpath ~/mongo-data --port 27017
```

MONGOD LOG (STRUCTURED JSON)

```
{"t": {"$date": "2026-09-27T10:15:02+03:00"},
 "s": "I", "c": "NETWORK", "id": 23016,
 "msg": "Waiting for connections",
 "attr": {"port": 27017, "ssl": "off"}}
```

Platform	Run as a service	Configuration file	Data directory
Windows (MSI)	service *MongoDB*, starts automatically	`…\Server\8.0\bin\mongod.cfg`	`…\Server\8.0\data`
macOS (Homebrew)	`brew services start mongodb-community`	`/opt/homebrew/etc/mongod.conf`	`/opt/homebrew/var/mongodb`
Linux (`apt` / `dnf`)	`sudo systemctl start mongod`	`/etc/mongod.conf`	`/var/lib/mongodb` or `/var/lib/mongo`

MongoDB Atlas: A Free Cloud Cluster



Create
a free **M0** cluster

Add a user
database user and password

Allow access
your IP address

Copy the URI
`mongodb+srv://…`

Connect
`mongosh`, Compass or Python

atlas_ping.py

```python
import os

from pymongo import MongoClient

uri = os.environ["MONGODB_URI"]   # never hard-code passwords
with MongoClient(uri) as client:
    print(client.admin.command("ping"))
```

SAMPLE OUTPUT

```
{'ok': 1}
```


The MongoDB Shell: mongosh

- **`mongosh`** is a JavaScript and Node.js **REPL** for MongoDB
- It replaced the legacy `mongo` shell, which was removed in MongoDB 6.0
- Use it for CRUD, administration, aggregation and indexing
- Syntax highlighting, auto-completion (Tab) and `help` built in

TERMINAL

```
$ mongosh "mongodb://127.0.0.1:27017/world"
$ mongosh "mongodb://localhost:27017/world" --file exercise03.js
$ mongosh "mongodb+srv://cluster0.example.mongodb.net/world" --username ada
$ mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"
1
```

Tip

With Docker no local install is needed: `docker exec -it mongo mongosh` starts the shell inside the container.

mongosh Is a JavaScript REPL

mongosh

```
test> Math.sin(Math.PI / 4)
0.7071067811865475
test> Math.tan(Math.PI / 4)
0.9999999999999999
test> function fact(n) {
...   return n <= 1 ? 1 : n * fact(n - 1);
... }
test> fact(5)
120
```

```
>>> import math
>>> math.tan(math.pi / 4)
0.9999999999999999
>>> def fact(n: int) -> int:
...     return 1 if n <= 1 else n * fact(n - 1)
...
>>> fact(5)
120
```

Note

Both languages use IEEE-754 double-precision floats, so both print `0.9999999999999999` for tan(π/4).

Basic mongosh Commands

Command	Effect
`show dbs`	List the databases and their sizes
`use world`	Switch to database `world` (created lazily on first write)
`db`	Print the name of the current database
`show collections`	List the collections of the current database
`show users`	List the users defined in the current database
`db.countries1.findOne()`	Show one document of a collection
`help`, `db.countries1.help()`	Help for the shell and for a collection
`exit`	Leave the shell

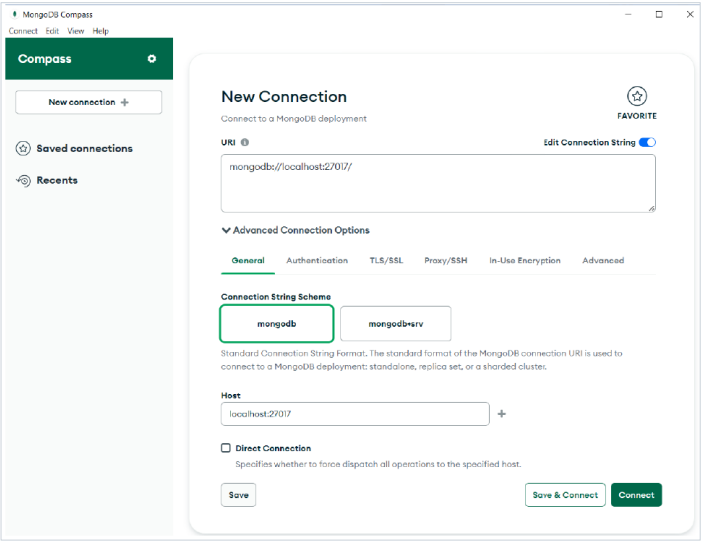
MongoDB Command-Line Tools

Tool	Purpose
`mongod`	The database server
`mongos`	Query router of a sharded cluster
`mongosh`	The shell (the legacy `mongo` shell was removed in 6.0)
`mongodump` / `mongorestore`	Binary (BSON) backup and restore
`mongoexport` / `mongoimport`	Export and import JSON, CSV or TSV — not for backups
`bsondump`	Convert BSON dump files to JSON for inspection
`mongostat` / `mongotop`	Live server counters / time spent per collection
`mongofiles`	Store and retrieve files in **GridFS**

Removed from the original list: `mongooplog`, `mongoperf` *and* `mongosniff` *no longer exist;* `mongorestore` *was listed twice.*

MongoDB Compass

- Free **GUI** for MongoDB
- Browse databases, collections and documents
- Build filters, projections and sorts visually
- Aggregation pipeline builder and schema analysis
- Manage indexes and read `explain` plans
- Built-in `mongosh` tab



Compass: the New Connection dialog

MINI LAB

Set Up Your MongoDB Environment

1. Start MongoDB 8 (the Docker command or a local install)
2. Connect with `mongosh` and run `db.runCommand({ ping: 1 })`
3. Run `use ain3005`, then `show dbs` — is `ain3005` listed?
4. Insert `{ name: "<your name>" }` into `db.students`, then `show dbs` again
5. Open Compass, connect to `mongodb://localhost:27017` and find your document

> **Note**
> Keep this setup: every exercise in the rest of the module uses it.

QUIZ

Quick Check: Tools

You need a backup of the `shop` database that preserves every BSON type. Which tool do you use?

A. `mongoexport`

B. `mongodump`

C. `mongostat`

D. `mongosh` with `show dbs`

ANSWER

Quick Check: Tools

You need a backup of the `shop` database that preserves every BSON type. Which tool do you use?

- A `mongoexport`
- **B** `mongodump`
- C `mongostat`
- D `mongosh` with `show dbs`

Why

`mongodump` writes the documents as BSON, so all types survive `mongorestore`. `mongoexport` writes JSON or CSV and can lose type information.

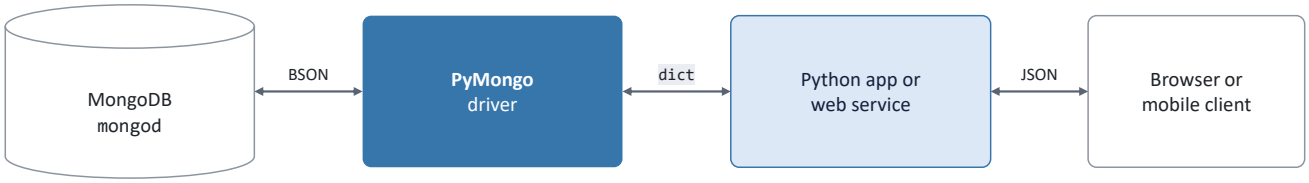
04

PyMongo Basics

Connecting from Python: clients, databases, collections and cursors

- BSON on the wire, `dict` in Python
- Installing PyMongo and connecting with `MongoClient`
- Connection strings, databases and collections
- Cursors — and eight exercises from the original deck

From BSON on the Wire to Python Objects



- **BSON** = binary JSON: typed, length-prefixed, fast to scan
- The driver encodes `dict` → BSON and decodes BSON → `dict`
- A web API re-encodes results as JSON for its clients (Module 8)

```
>>> import bson
>>> data = bson.encode({"name": "Ada"})
>>> data
b'\x13\x00\x00\x00\x02name\x00\x04\x00\x00\x00Ada\x00\x00'
>>> len(data)
19
>>> bson.decode(data)
{'name': 'Ada'}
```


Installing PyMongo

TERMINAL

```
$ python -m venv .venv
$ source .venv/bin/activate    # Windows: .venv\Scripts\activate
$ python -m pip install pymongo
$ python -c "import pymongo; print(pymongo.version)"
4.17.0
```

- One package, `pymongo`: includes the `bson` and `gridfs` modules
- PyMongo 4.x has a synchronous **and** an asynchronous API (`AsyncMongoClient`, Section 11)
- Motor, the former async driver, is deprecated

> ! **Watch out**
> Never `pip install bson`: that unrelated package shadows PyMongo's `bson` module and breaks imports.

4 · PYMONGO BASICS

Connecting with MongoClient

connect.py

```python
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure

URI = "mongodb://localhost:27017"


def main() -> None:
    with MongoClient(URI, serverSelectionTimeoutMS=3000) as client:
        try:
            client.admin.command("ping")
        except ConnectionFailure as exc:
            raise SystemExit(f"MongoDB unreachable: {exc}")
        print(client.list_database_names())


if __name__ == "__main__":
    main()
```

SAMPLE OUTPUT

```
['admin', 'config', 'local', 'world']
```

- `MongoClient` manages a thread-safe **connection pool**
- Create **one** client per application and reuse it
- Creating it does not block: it connects in the background
- `ping` fails fast; the default wait is 30 s
- `with` closes the pool (context managers: Module 4)

4 · PYMONGO BASICS

Connection Strings

connection string (URI)

```
mongodb://ada:s3cret@db1:27017,db2:27017/shop?replicaSet=rs0&authSource=admin
```

Part	Example	Meaning
Scheme	`mongodb://` or `mongodb+srv://`	Explicit host list, or DNS seed list (Atlas)
Credentials	`ada:s3cret@`	User and password, percent-encoded
Hosts	`db1:27017,db2:27017`	One or more members; the default port is 27017
Database	`/shop`	Default database for `get_default_database()`
Options	`?replicaSet=rs0&authSource=admin`	Replica set name, auth database, timeouts, TLS

```
>>> from urllib.parse import quote_plus
>>> quote_plus("p@ss:w/rd")
'p%40ss%3Aw%2Frd'
```

Tip

Characters such as `@ : /` in a password must be percent-encoded. Better: keep the whole URI in an environment variable.

Databases and Collections in PyMongo

handles.py

```python
from pymongo import MongoClient

with MongoClient("mongodb://localhost:27017") as client:
    db = client["world"]            # or: client.world
    countries = db["countries1"]    # or: db.countries1

    print(db.name)
    print(countries.full_name)
    print(client.world.countries1 == countries)
```

OUTPUT

```
world
world.countries1
True
```

- Item access `client["world"]` or attribute access `client.world`
- Prefer item access for names like `"my-db"` or names that clash with methods
- Getting a handle is free: **no network round trip**
- Databases and collections are created by the first write

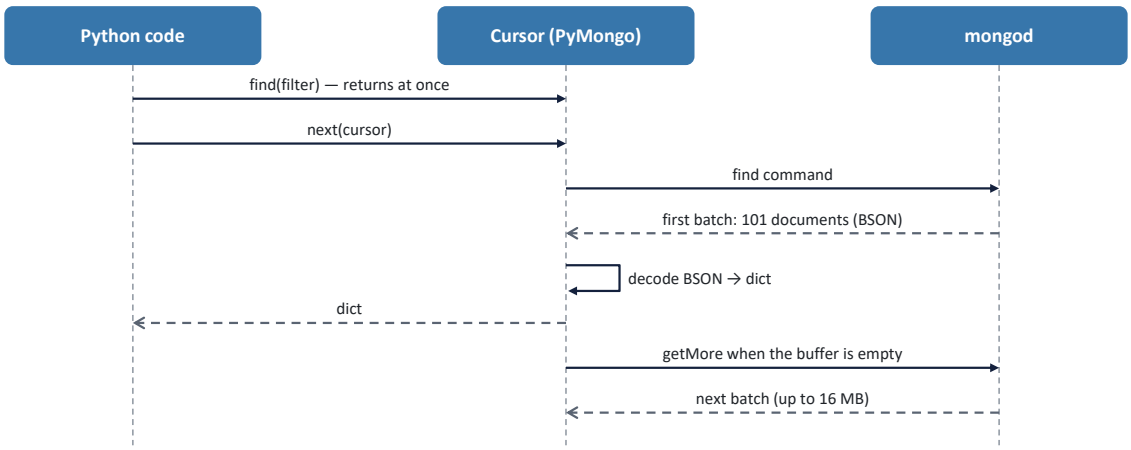
Translating mongosh to PyMongo

Aspect	mongosh (JavaScript)	PyMongo (Python)
Method names	`insertOne`, `findOne`, `updateMany`	`insert_one`, `find_one`, `update_many`
Keys	`{name: 1}` — quotes optional	`{"name": 1}` — quotes required
Operators	`{$gt: 5}`	`{"$gt": 5}`
Literals	`true`, `false`, `null`	`True`, `False`, `None`
Sorting	`.sort({year: -1})`	`.sort("year", -1)` or a list of pairs
Ids and dates	`ObjectId("…")`, `ISODate("…")`	`bson.ObjectId("…")`, aware `datetime`
Results	`{acknowledged: true, insertedId: 101}`	`result.inserted_id`

Key idea

Rule of thumb: camelCase becomes snake_case, and every key and $-operator becomes a quoted string.

Cursors: How Query Results Arrive



Key idea

A cursor is **lazy** and **single-pass**: iterate it once, or call `list(cursor)` to keep the results.

TRY IT

Browse the world Database

The `world` database contains the collection `countries1`. Write a script that:

1. connects to `mongodb://localhost:27017` in a `with` block
2. pretty-prints every document of `countries1`
3. prints the sorted list of distinct `continent` values

Tip

Hint: `find()` returns an iterable cursor; `distinct(field)` returns a Python `list`.

ANSWER

Browse the world Database

browse_world.py

```python
from pprint import pprint

from pymongo import MongoClient

with MongoClient("mongodb://localhost:27017") as client:
    countries = client["world"]["countries1"]

    for country in countries.find():
        pprint(country)

    pprint(sorted(countries.distinct("continent")))
```

SAMPLE OUTPUT (LAST LINE)

```
['Africa', 'Asia', 'Europe', 'North America', 'Oceania', 'South America']
```


TRACE THE CODE

Read an Aggregation Pipeline

`movies1` stores each movie's genres as an array of sub-documents. What does the script print for these two movies?

imdb.movies1

```
[
  {"title": "Alien",
   "genres": [{"name": "Horror"},
              {"name": "Sci-Fi"}]},
  {"title": "Arrival",
   "genres": [{"name": "Sci-Fi"},
              {"name": "Drama"}]}
]
```

```python
pipeline = [
    {"$project": {"genres": 1}},
    {"$unwind": "$genres"},
    {"$group": {"_id": "$genres.name",
                "total": {"$sum": 1}}},
]
for row in movies.aggregate(pipeline):
    print(row)
```

ANSWER

Read an Aggregation Pipeline

OUTPUT (ORDER MAY VARY)

```
{'_id': 'Sci-Fi', 'total': 2}
{'_id': 'Horror', 'total': 1}
{'_id': 'Drama', 'total': 1}
```

- `$project` keeps only `_id` and `genres`
- `$unwind` emits one document per genre: 4 documents
- `$group` counts the documents per genre name
- Add `{"$sort": {"total": -1}}` for a fixed order

FIND THE BUG

Why Does the 1970s Filter Fail?

The script should list movies from the 1970s, newest first, showing only title and year. It prints movies from **every** decade before 1980. Why?

seventies.py

```python
from pymongo import DESCENDING, MongoClient

with MongoClient("mongodb://localhost:27017") as client:
    movies = client["imdb"]["movies1"]
    query = {"year": {"$gte": 1970},
             "year": {"$lt": 1980}}
    projection = {"_id": 0, "title": 1, "year": 1}
    cursor = movies.find(query, projection).sort(
        [("year", DESCENDING), ("title", 1)])
    for movie in cursor:
        print(movie)
```

ANSWER

Why Does the 1970s Filter Fail?

- A dict literal cannot hold `"year"` twice: Python keeps the **last** entry
- The filter that reaches MongoDB is `{"year": {"$lt": 1980}}`
- Fix: put both operators in **one** sub-document

fixed line 5

```
query = {"year": {"$gte": 1970, "$lt": 1980}}
```

> **Note**
> The original example also created two `MongoClient` objects and used `"year": 2` in the projection — any non-zero value means *include*, but write `1`.

TRACE THE CODE

Which Country Names Match?

The original example finds countries whose name matches `^.{5}$`. Predict the output of this pure-Python version, then write the PyMongo query.

```
import re

pattern = re.compile(r"^.{5}$")
names = ["Chad", "Chile", "China", "Cuba", "Egypt",
         "Peru", "Spain", "Turkey"]

print([name for name in names if pattern.match(name)])
```

ANSWER

Which Country Names Match?

OUTPUT

```
['Chile', 'China', 'Egypt', 'Spain']
```

PyMongo

```
pattern = re.compile(r"^.{5}$")
cursor = countries.find({"name": pattern},
                        {"_id": 0, "name": 1})
for country in cursor:
    print(country)
```

- `^` and `$` anchor the whole name; `.{5}` means exactly five characters
- PyMongo sends a compiled `re.Pattern` as a BSON regular expression

TRY IT

Replace the Removed group() Method

These legacy examples call `Collection.group()`, which was removed in MongoDB 4.2 and PyMongo 4. Rewrite both with `aggregate()` and a `$group` stage.

legacy code — fails with PyMongo 4

```
# Legacy: number of countries per continent (Example 6)
countries.group(key={"continent": 1}, condition={},
                initial={"count": 0},
                reduce="function(c, h) { h.count += 1; }")

# Legacy: country names per continent (Example 8)
countries.group(key={"continent": 1}, condition={},
                initial={"countries": []},
                reduce="function(c, h) { h.countries.push(c.name); }")
```

Tip

Hint: `{"$sum": 1}` counts documents; `{"$push": "$name"}` collects values into an array.

ANSWER

Replace the Removed group() Method

PyMongo 4

```
counts = countries.aggregate([
    {"$group": {"_id": "$continent", "count": {"$sum": 1}}},
    {"$sort": {"count": -1}},
])
for row in counts:
    print(row["_id"], row["count"])

names = countries.aggregate([
    {"$group": {"_id": "$continent",
                "countries": {"$push": "$name"}}},
])
```

Key idea

Operators such as `$sum` and `$push` replace server-side JavaScript: faster, safer and supported by every driver.

TRY IT

Replace a $where Query

Example 7 finds movies with more than one director using `$where`, which runs JavaScript for every document and is deprecated since MongoDB 8.0. Rewrite the filter without JavaScript.

legacy code

```
movies.find({"$where": "this.directors.length > 1"},
            {"_id": 0, "title": 1, "directors": 1})
```

Tip

Hint: dot notation can address an array position — `"directors.1"` is the **second** element.

ANSWER

Replace a $where Query

PyMongo

```
projection = {"_id": 0, "title": 1, "directors": 1}

# a second director exists
by_position = {"directors.1": {"$exists": True}}

# alternative: an aggregation expression
by_size = {"$expr": {"$gt": [{"$size": "$directors"}, 1]}}

for movie in movies.find(by_position, projection):
    print(movie)
```

- `directors.1` exists only if the array has at least two elements
- `$size` inside `$expr` fails on documents without a `directors` array
- `$where` cannot use indexes and executes JavaScript per document

05

CRUD Operations

Create, read, update and delete — in mongosh and in PyMongo

- `insert_one`, `insert_many` and duplicate keys
- `find_one` and `find` for simple reads
- `update_one`, `update_many`, update operators, `replace_one`
- `delete_one`, `delete_many` and result objects

5 · CRUD OPERATIONS

CRUD at a Glance

Operation	mongosh	PyMongo	Returns (PyMongo)
Create	`insertOne` `insertMany`	`insert_one` `insert_many`	`InsertOneResult.inserted_id` `InsertManyResult.inserted_ids`
Read	`findOne` `find`	`find_one` `find`	`dict` or `None` `Cursor`
Update	`updateOne`, `updateMany` `replaceOne`	`update_one`, `update_many` `replace_one`	`UpdateResult.matched_count` `UpdateResult.modified_count`
Delete	`deleteOne` `deleteMany`	`delete_one` `delete_many`	`DeleteResult.deleted_count`

Setup

The PyMongo snippets in this section assume `db = client["shop"]` and the collection handles `customers`, `products` and `orders`.

5 · CRUD OPERATIONS

Creating Databases and Collections

mongosh

```
test> use shop
switched to db shop
shop> db
shop
shop> db.createCollection("customers")
{ ok: 1 }
shop> show collections
customers
```

PyMongo

```
db = client["shop"]            # no server call
print(db.list_collection_names())

db.create_collection("customers")
print(db.list_collection_names())
```

OUTPUT

```
[]
['customers']
```

Key idea

Databases and collections are created **lazily** by the first insert. Call `create_collection` only when you need options: validation, capped or time-series collections.

Inserting One Document

Insert customer 101: John Doe, age 30, from New York.

mongosh

```
db.customers.insertOne({
  _id: 101,
  name: "John Doe",
  age: 30,
  city: "New York"
})
```

OUTPUT

```
{ acknowledged: true, insertedId: 101 }
```

PyMongo

```
result = customers.insert_one({
    "_id": 101,
    "name": "John Doe",
    "age": 30,
    "city": "New York",
})
print(result.inserted_id)
```

OUTPUT

```
101
```

Watch out

Omit `_id` and the driver generates an `ObjectId` — and adds it to *your* `dict` (Section 7).

Inserting Many Documents

mongosh

```
db.products.insertMany([
  { _id: 101, name: "Product A",
    price: 15, category: "Electronics" },
  { _id: 102, name: "Product B",
    price: 20, category: "Clothing" }
])
```

OUTPUT

```
{ acknowledged: true,
  insertedIds: { '0': 101, '1': 102 } }
```

PyMongo

```
result = products.insert_many([
    {"_id": 101, "name": "Product A",
     "price": 15, "category": "Electronics"},
    {"_id": 102, "name": "Product B",
     "price": 20, "category": "Clothing"},
])
print(result.inserted_ids)
```

OUTPUT

```
[101, 102]
```

Tip

`ordered=True` (default) stops at the first error; `insert_many(docs, ordered=False)` inserts all valid documents and then raises `BulkWriteError`.

5 · CRUD OPERATIONS

Handling Duplicate Keys

PyMongo

```python
from pymongo.errors import DuplicateKeyError

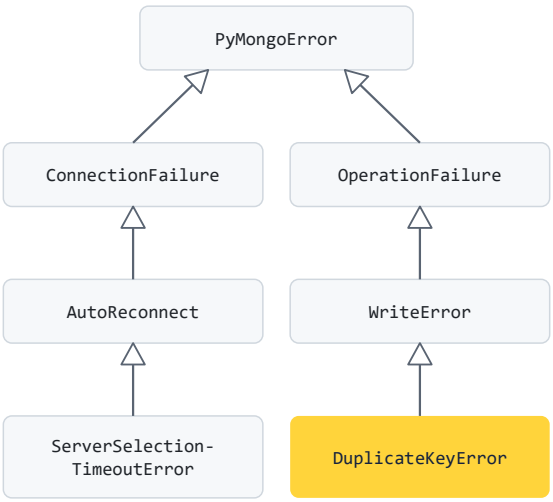
try:
    customers.insert_one(
        {"_id": 101, "name": "Ada Lovelace"})
except DuplicateKeyError as exc:
    print("customer 101 exists:",
          exc.details["keyValue"])
```

OUTPUT

```
customer 101 exists: {'_id': 101}
```

Tip

Catch the most specific exception you can handle and let the others propagate (Module 1).



FIND THE BUG

Why Does the Second Insert Fail?

Only the pencil is stored, then `DuplicateKeyError` is raised. Why?

add_products.py

```python
template = {"category": "Stationery", "price": 2}

for name in ["Pencil", "Eraser", "Ruler"]:
    template["name"] = name
    products.insert_one(template)
```

Tip

Hint: print `template` after the first `insert_one` call.

ANSWER

Why Does the Second Insert Fail?

- `insert_one` adds the generated `_id` to the dict you pass
- Iteration 2 sends the **same** `_id` again → `DuplicateKeyError`
- Fix: insert a fresh dict each time, or one `insert_many`

fixed

```
products.insert_many(
    [{**template, "name": name}
     for name in ["Pencil", "Eraser", "Ruler"]])
```


5 · CRUD OPERATIONS

Reading Documents: find_one and find

mongosh

```
db.customers.findOne({ _id: 101 })

db.customers.find({ city: "New York" })

db.customers.countDocuments(
  { city: "New York" })
```

PyMongo

```
customer = customers.find_one({"_id": 101})
if customer is None:
    print("no such customer")

for doc in customers.find({"city": "New York"}):
    print(doc["name"])

print(customers.count_documents(
    {"city": "New York"}))
```

- `find_one` returns a `dict` or `None`; `find` returns a lazy `Cursor`
- `Cursor.count()` was removed in PyMongo 4: use `count_documents(filter)`
- Filters, projections, sorting and pagination: Section 8

5 · CRUD OPERATIONS

Updating One Document with $set

Change the age of customer 101 to 35.

mongosh

```
db.customers.updateOne(
  { _id: 101 },
  { $set: { age: 35 } }
)
```

OUTPUT

```
{ acknowledged: true, insertedId: null,
  matchedCount: 1, modifiedCount: 1,
  upsertedCount: 0 }
```

PyMongo

```
result = customers.update_one(
    {"_id": 101},
    {"$set": {"age": 35}},
)
print(result.matched_count,
      result.modified_count)
```

OUTPUT

```
1 1
```

Key idea

Run it twice: `matched_count` stays 1 but `modified_count` becomes 0 — the value is already 35.

5 · CRUD OPERATIONS

Updating Many Documents

Set the price of every product in the Electronics category to 20.

mongosh

```
db.products.updateMany(
  { category: "Electronics" },
  { $set: { price: 20 } }
)
```

PyMongo

```
result = products.update_many(
    {"category": "Electronics"},
    {"$set": {"price": 20}},
)
print(result.modified_count)

# raise every clothing price by 5
products.update_many({"category": "Clothing"},
                     {"$inc": {"price": 5}})
```

Watch out

An empty filter `{}` matches **every** document — double-check the filter before `update_many`.

5 · CRUD OPERATIONS

Update Operators

Operator	Example (PyMongo)	Effect
`$set`	`{"$set": {"city": "Izmir"}}`	Set or add fields
`$unset`	`{"$unset": {"age": ""}}`	Remove a field
`$inc`	`{"$inc": {"stock": -1}}`	Add to a number
`$mul`	`{"$mul": {"price": 1.1}}`	Multiply a number
`$rename`	`{"$rename": {"zip": "postcode"}}`	Rename a field
`$push`	`{"$push": {"tags": "vip"}}`	Append to an array
`$addToSet`	`{"$addToSet": {"tags": "vip"}}`	Append if not already present
`$pull`	`{"$pull": {"tags": "vip"}}`	Remove matching array elements
`$currentDate`	`{"$currentDate": {"updated": True}}`	Set to the server's current date

Tip

Combine operators in one update document: `{"$set": {...}, "$inc": {...}}` — the whole document changes atomically.

QUIZ

Quick Check: Update Results

`products` contains five Electronics items; two of them already cost 20. What does this print?

```
result = products.update_many(
    {"category": "Electronics"},
    {"$set": {"price": 20}},
)
print(result.matched_count, result.modified_count)
```

A `5 5`

B `5 3`

C `3 3`

D `2 3`

ANSWER

Quick Check: Update Results

`products` contains five Electronics items; two of them already cost 20. What does this print?

```
result = products.update_many(
    {"category": "Electronics"},
    {"$set": {"price": 20}},
)
print(result.matched_count, result.modified_count)
```

A `5 5`

B `5 3`

C `3 3`

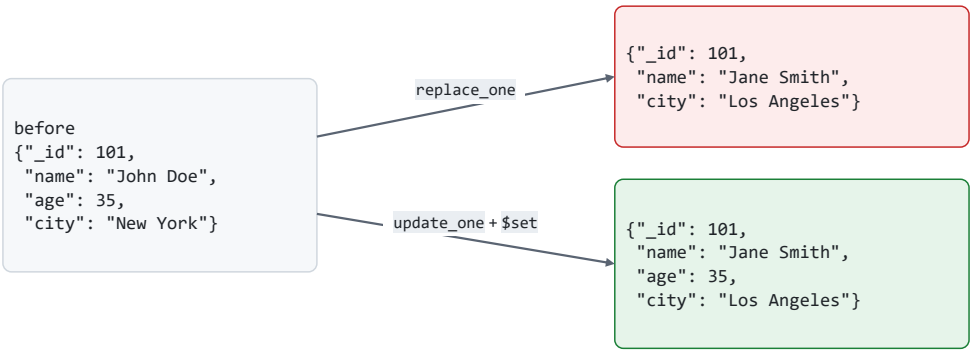
D `2 3`

Why

All five documents match the filter, but only the three whose price differs are modified.

5 · CRUD OPERATIONS

Replacing a Document with replace_one



PyMongo

```
customers.replace_one(
    {"name": "John Doe"},
    {"name": "Jane Smith", "city": "Los Angeles"},
)
```

Watch out

`replace_one` keeps only `_id`: omitted fields (here `age`) are gone. Updates need `$` operators, replacements must not contain them — else `ValueError`.

5 · CRUD OPERATIONS

Update and Return: find_one_and_update

mongosh

```
db.customers.findOneAndUpdate(
  { name: "John Doe" },
  { $set: { name: "Jane Smith",
            city: "Los Angeles" } },
  { returnNewDocument: true }
)
```

PyMongo

```
from pymongo import ReturnDocument

updated = customers.find_one_and_update(
    {"name": "John Doe"},
    {"$set": {"name": "Jane Smith",
              "city": "Los Angeles"}},
    return_document=ReturnDocument.AFTER,
)
print(updated)
```

SAMPLE OUTPUT

```
{'_id': 101, 'name': 'Jane Smith', 'age': 35,
 'city': 'Los Angeles'}
```

- Finds, updates and **returns** one document in a single atomic step
- Returns the document *before* the update unless you ask for `AFTER`; `None` if nothing matched
- Siblings: `find_one_and_replace`, `find_one_and_delete`

5 · CRUD OPERATIONS

Adding to Arrays with $push

Append a new line item to the `items` array of order 1001.

mongosh

```
db.orders.updateOne(
  { _id: 1001 },
  { $push: {
      items: { product_id: 103,
               quantity: 3, price: 10 }
    }
  }
)
```

PyMongo

```
line = {"product_id": 103, "quantity": 3,
        "price": 10}
orders.update_one({"_id": 1001},
                  {"$push": {"items": line}})

# several items at once
orders.update_one(
    {"_id": 1001},
    {"$push": {"items": {"$each": [line_a, line_b]}}},
)
```

Tip

`$addToSet` skips values that are already present; `$pull` removes matching elements. Keep arrays bounded — a document cannot exceed 16 MB.

Upserts: Update or Insert

PyMongo

```python
from datetime import UTC, datetime

result = customers.update_one(
    {"email": "ada@example.com"},
    {"$set": {"name": "Ada Lovelace"},
     "$setOnInsert": {"joined": datetime.now(UTC)}},
    upsert=True,
)
print(result.matched_count, result.upserted_id)
```

SAMPLE OUTPUT

```
0 ObjectId('66f6a2c1e4b0c95f8a1d2e3f')    # first run
1 None                                    # second run
```

- No match → a new document is built from the filter plus the update
- `$setOnInsert` applies only when a document is inserted
- `upserted_id` is `None` when an existing document was updated

Deleting Documents

mongosh

```
db.customers.deleteOne({ _id: 101 })

db.products.deleteMany(
  { price: { $lt: 10 } })
```

SAMPLE OUTPUT

```
{ acknowledged: true, deletedCount: 1 }
{ acknowledged: true, deletedCount: 4 }
```

PyMongo

```python
result = customers.delete_one({"_id": 101})
print(result.deleted_count)

result = products.delete_many(
    {"price": {"$lt": 10}})
print(result.deleted_count)
```

SAMPLE OUTPUT

```
1
4
```

Watch out

`delete_many({})` removes **every** document but keeps the collection and its indexes; `products.drop()` removes the collection itself.

Wrapping CRUD in a Repository Class

customers.py

```python
from dataclasses import asdict, dataclass

from pymongo.collection import Collection


@dataclass
class Customer:
    _id: int
    name: str
    city: str
```

```python
class CustomerRepository:
    """All MongoDB access for customers."""

    def __init__(self, collection: Collection) -> None:
        self._customers = collection

    def add(self, customer: Customer) -> None:
        self._customers.insert_one(asdict(customer))

    def get(self, customer_id: int) -> Customer | None:
        doc = self._customers.find_one({"_id": customer_id})
        return Customer(**doc) if doc else None
```

```python
repo = CustomerRepository(client["shop"]["customers"])
repo.add(Customer(101, "John Doe", "New York"))
print(repo.get(101))
```

OUTPUT

```
Customer(_id=101, name='John Doe', city='New York')
```


TRY IT

A Complete CRUD Script

Write `crud_demo.py` that works on the `customers` collection of database `shop`:

1. Insert customers 201 (Ada, London) and 202 (Linus, Helsinki) with **one** call
2. Move Linus to Portland and print `modified_count`
3. Add the tag `"vip"` to Ada's `tags` array without creating duplicates
4. Delete every customer in London and print `deleted_count`
5. Print how many customers remain

Tip

Use a `with MongoClient(...)` block and the result objects of each call.

ANSWER

A Complete CRUD Script

crud_demo.py

```python
from pymongo import MongoClient

with MongoClient("mongodb://localhost:27017") as client:
    customers = client["shop"]["customers"]
    customers.insert_many([
        {"_id": 201, "name": "Ada", "city": "London"},
        {"_id": 202, "name": "Linus", "city": "Helsinki"},
    ])
    moved = customers.update_one(
        {"_id": 202}, {"$set": {"city": "Portland"}})
    print("modified:", moved.modified_count)
    customers.update_one(
        {"_id": 201}, {"$addToSet": {"tags": "vip"}})
    removed = customers.delete_many({"city": "London"})
    print("deleted:", removed.deleted_count)
    print("remaining:", customers.count_documents({}))
```


06

Data Types and Schema Design

BSON types in mongosh and Python, and enforcing structure with $jsonSchema

- From JSON to BSON: the types MongoDB stores
- Numbers, dates, arrays and embedded documents in PyMongo
- Schema validation with `$jsonSchema`

From JSON to BSON

JSON (6 types)	BSON adds, for example
string	32- and 64-bit integers, `decimal128`
number	date (UTC milliseconds)
boolean	`ObjectId`
null	binary data
array	regular expression
object	timestamp, min/max key

Key idea

BSON is a **binary**, length-prefixed format: typed, fast to traverse, at most **16 MB** per document.

PyMongo's `bson` package

```
>>> import bson
>>> data = bson.encode({"n": 1})
>>> data
b'\x0c\x00\x00\x00\x10n\x00\x01\x00\x00\x00\x00'
>>> len(data)
12
>>> bson.decode(data)
{'n': 1}
```


BSON Types in mongosh and Python

BSON type	mongosh literal	PyMongo (Python)
null	`null`	`None`
boolean	`true`, `false`	`bool`
double	`3.14`, also `3` (shell default)	`float`
int32 / int64	`NumberInt("3")`, `NumberLong("3")`	`int` (decoded int64: `bson.Int64`)
decimal128	`NumberDecimal("19.99")`	`bson.Decimal128`
string (UTF-8)	`"foobar"`	`str`
date	`new Date()`, `ISODate("2024-03-01")`	`datetime.datetime` (UTC)
regular expression	`/foobar/i`	`re.Pattern` → `bson.Regex`
array	`["a", "b", "c"]`	`list` (a `tuple` is stored as an array)
embedded document	`{ city: "Anytown" }`	`dict`
binary data	`BinData(0, "AP8=")`	`bytes` / `bson.Binary`
ObjectId	`ObjectId("5392…")`	`bson.ObjectId`

Numbers and Dates: Precision Matters

- mongosh numbers are 64-bit **doubles** unless you write `NumberInt`/`NumberLong`
- PyMongo stores `int` as int32 or int64 (by size) and `float` as double
- Money: use **decimal128** — binary floats cannot represent 0.1 exactly
- Dates are UTC milliseconds since 1970: the **time zone is not stored**

```python
from datetime import datetime, timezone
from decimal import Decimal

import bson

doc = {"qty": 3, "views": 2 ** 40, "price": 0.1 + 0.2,
       "exact": bson.Decimal128(Decimal("0.3")),
       "at": datetime(2024, 3, 1, 9, 30, tzinfo=timezone.utc)}

for key, value in bson.decode(bson.encode(doc)).items():
    print(f"{key:<6}{type(value).__name__:<11}{value}")
```

OUTPUT

```
qty   int         3
views Int64       1099511627776
price float       0.30000000000000004
exact Decimal128 0.3
at    datetime    2024-03-01 09:30:00
```


Arrays, Embedded Documents and Binary Data

PyMongo

```python
customer = {
    "name": "John Doe",
    "address": {
        "street": "123 Park Street",
        "city": "Anytown",
        "state": "NY",
    },
    "tags": ["vip", "newsletter"],
    "avatar": b"\x89PNG...",
}
customers.insert_one(customer)
customers.find_one({"address.city": "Anytown"})
customers.find_one({"tags": "vip"})
```

Embedded document

A `dict` inside a document; reach into it with **dot notation**: `"address.city"`

Array

A `list`; MongoDB understands arrays: query, update ($push) and index their elements

Binary data

`bytes`; the only way to store non-UTF-8 data. Files above 16 MB → **GridFS**

TRACE THE CODE

What Survives the Round Trip?

```python
from datetime import datetime, timedelta, timezone

import bson

istanbul = timezone(timedelta(hours=3))
doc = {
    "tags": ("mongo", "python"),
    "count": 3,
    "created": datetime(2024, 3, 1, 12, 30, 15, 123456,
                        tzinfo=istanbul),
}
back = bson.decode(bson.encode(doc))
print(type(back["tags"]).__name__, back["tags"])
print(back["count"] == doc["count"])
print(back["created"])
print(back["created"].tzinfo)
```


ANSWER

What Survives the Round Trip?

OUTPUT

```
list ['mongo', 'python']
True
2024-03-01 09:30:15.123000
None
```

- BSON has no tuple type: a `tuple` is stored as an array and read back as a `list`
- The time is converted to **UTC** (12:30 at UTC+3 → 09:30)
- Precision is **milliseconds**: 123456 µs becomes 123000 µs
- Without `tz_aware=True` the decoded `datetime` is naive: `tzinfo` is `None`

Schema Validation with $jsonSchema

create_students.py

```python
from pymongo import MongoClient

STUDENT_SCHEMA = {
    "bsonType": "object",
    "required": ["name", "year", "gpa"],
    "properties": {
        "name": {"bsonType": "string", "minLength": 1},
        "year": {"bsonType": "int", "minimum": 1, "maximum": 4},
        "gpa": {"bsonType": "number", "minimum": 0, "maximum": 4},
        "email": {"bsonType": "string", "pattern": "^[^@]+@[^@]+$"},
    },
}

with MongoClient("mongodb://localhost:27017") as client:
    client["university"].create_collection(
        "students", validator={"$jsonSchema": STUDENT_SCHEMA})
```

Key idea

A flexible schema is not *no* schema: the server checks every insert and update against the rules.

When a Document Breaks the Rules

```python
from pymongo.errors import WriteError

students = client["university"]["students"]
students.insert_one(
    {"name": "Ada", "year": 2, "gpa": 3.7})

try:
    students.insert_one(
        {"name": "Bob", "year": 7})
except WriteError as exc:
    print(exc.code, exc.details["errmsg"])
```

SAMPLE OUTPUT

```
121 Document failed validation
```

Option	Values
`validationLevel`	`"strict"` (default): every insert and update `"moderate"`: skip documents that were already invalid
`validationAction`	`"error"` (default): reject the write `"warn"`: accept it and write a log entry

existing collection

```python
db.command("collMod", "students",
           validator={"$jsonSchema": schema})
```

TRY IT

Write a Validator for Products

Create a `products` collection whose documents must have a non-empty `name`, a `price` ≥ 0 and an integer `stock` ≥ 0. Optional `tags` must be an array of strings.

tests

```
products.insert_one({"name": "Pen", "price": 1.5, "stock": 40})  # ok
products.insert_one({"name": "Pad", "price": -2, "stock": 5})  # fails
products.insert_one({"name": "Ink", "price": 3, "stock": 9,
                     "tags": [1, 2]})  # fails
```

Tip

Hint: an array field uses `"bsonType": "array"` with an `"items"` schema for its elements.

ANSWER

Write a Validator for Products

```
PRODUCT_SCHEMA = {
    "bsonType": "object",
    "required": ["name", "price", "stock"],
    "properties": {
        "name": {"bsonType": "string", "minLength": 1},
        "price": {"bsonType": "number", "minimum": 0},
        "stock": {"bsonType": "int", "minimum": 0},
        "tags": {
            "bsonType": "array",
            "items": {"bsonType": "string"},
        },
    },
}
db.create_collection(
    "products", validator={"$jsonSchema": PRODUCT_SCHEMA})
```

07

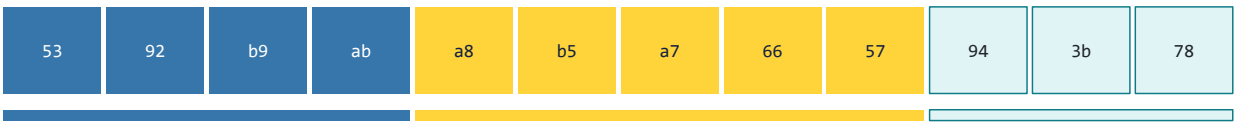
ObjectIds and Relations

Identity, creation time and linking documents

- The `_id` field and the 12-byte `ObjectId`
- Working with `bson.ObjectId` in Python
- Embedding vs referencing, and joins with `$lookup`

The _id Field and the ObjectId

- Every document has an `_id`: **unique** in its collection (automatic `_id_` index) and **immutable**
- Any BSON type except an array; if you omit it, the driver generates an `ObjectId`
- An `ObjectId` is 12 bytes, created by the client **without asking the server**



timestamp · 4 bytes
seconds since 1970
→ 2014-06-07 07:05:15 UTC

random value · 5 bytes
chosen once per process

counter · 3 bytes
increments per id

ObjectId("5392b9aba8b5a76657943b78") — all numbers are stored big-endian

Creating and Inspecting ObjectIds

mongosh

```
> db.world.findOne()._id
ObjectId('5392b9aba8b5a76657943b78')
> db.world.findOne()._id.getTimestamp()
ISODate('2014-06-07T07:05:15.000Z')
> db.world.findOne()._id.toString()
'5392b9aba8b5a76657943b78'
> new ObjectId()
ObjectId('66f6c0a1e4b0c2a3d4e5f601')
```

PyMongo (bson)

```
from bson import ObjectId

oid = ObjectId("5392b9aba8b5a76657943b78")
print(oid.generation_time)
print(str(oid))
print(ObjectId.is_valid("TUR"))
print(ObjectId() > oid)  # created now
```

OUTPUT

```
2014-06-07 07:05:15+00:00
5392b9aba8b5a76657943b78
False
True
```

Watch out

Convert with `str(oid)` for JSON and URLs, and back with `ObjectId(text)` before querying: the string `"5392…"` never equals the `ObjectId`.

TRACE THE CODE

Predict the ObjectId Facts

```
1  from bson import ObjectId
2
3  first = ObjectId("5392be94a8b5a76657943b7a")
4  second = ObjectId("5392be95a8b5a76657943b7b")
5
6  print(first < second)
7  gap = second.generation_time - first.generation_time
8  print(gap.total_seconds())
9  print(len(str(first)), len(first.binary))
10 print(first.binary[-3:].hex())
```

ANSWER

Predict the ObjectId Facts

OUTPUT

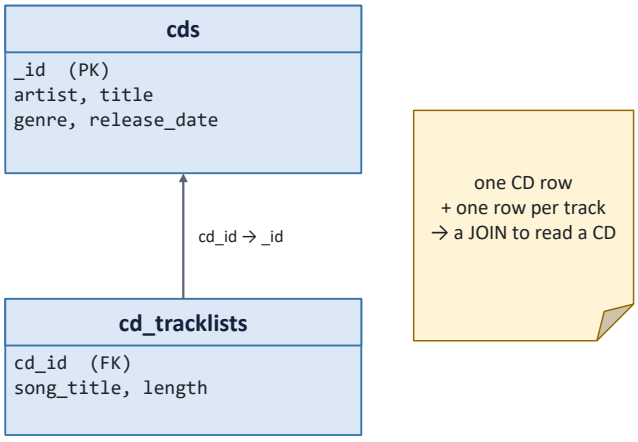
```
True
1.0
24 12
943b7a
```

- Ids compare byte by byte, so the timestamp decides: `first < second`
- Timestamps `...be94` and `...be95` differ by one second
- 24 hex characters encode the 12 raw bytes (`binary`)
- The last 3 bytes are the **counter**: `94 3b 7a`

7 · OBJECTIDS AND RELATIONS

Relations: Two Ways to Model a CD

Relational: two tables, joined by `cd_id`



Document: one CD, tracks embedded

media.items

```
{
  "type": "CD",
  "artist": "Nirvana",
  "title": "Nevermind",
  "genre": "Grunge",
  "release_date": "1991-09-24",
  "tracklist": [
    {"track": 1, "length": "5:02",
     "title": "Smells Like Teen Spirit"},
    {"track": 2, "length": "4:15",
     "title": "In Bloom"}
  ]
}
```

Embed or Reference?

	Embed	Reference
Relationship	one-to-one, one-to-few, *contains*	one-to-many (unbounded), many-to-many
Access	read together with the parent	read or updated on its own
Size	bounded, far below 16 MB	grows without limit
Writes	one atomic single-document write	several writes (or a transaction)
Reads	one query	a second query or `$lookup`

Key idea

Rule of thumb: **embed whenever you can** — every reference costs another query. Reference data that is unbounded, shared or accessed on its own.

Referencing in PyMongo

PyMongo

```
shop = client["shop"]
jane_id = shop.customers.insert_one(
    {"name": "Jane Smith", "city": "Izmir"}).inserted_id
shop.orders.insert_many([
    {"customer_id": jane_id, "product": "Phone", "quantity": 1},
    {"customer_id": jane_id, "product": "Case", "quantity": 2},
])

# following the reference costs a second query
order = shop.orders.find_one({"product": "Phone"})
buyer = shop.customers.find_one({"_id": order["customer_id"]})
print(buyer["name"])
```

Tip

Store the referenced `_id` itself (a *manual reference*) and index the field: `orders.create_index("customer_id")`.

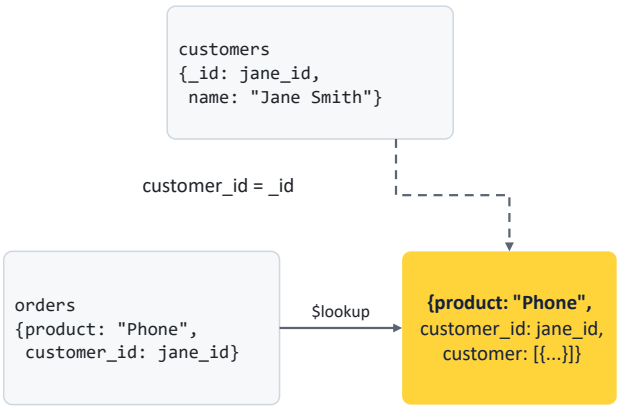
Joining References with $lookup

PyMongo

```
pipeline = [
    {"$match": {"product": "Phone"}},
    {"$lookup": {
        "from": "customers",
        "localField": "customer_id",
        "foreignField": "_id",
        "as": "customer",
    }},
    {"$unwind": "$customer"},
    {"$project": {"_id": 0, "product": 1,
                  "buyer": "$customer.name"}},
]
print(list(shop.orders.aggregate(pipeline)))
```

SAMPLE OUTPUT

```
[{'product': 'Phone', 'buyer': 'Jane Smith'}]
```



`$lookup` always produces an **array**; `$unwind` turns a one-element array into a sub-document.

DISCUSS

Embed or Reference? You Decide

For each scenario, decide whether to **embed** or to **reference**, and give one reason.

#	Scenario
1	A user's one to three shipping addresses
2	Reviews of a popular product (thousands, still growing)
3	Students and the courses they are enrolled in
4	The line items of an order, with the price actually paid
5	The author of a blog post, shown on every post page

ANSWER

Embed or Reference? You Decide

#	Choice	Why
1	**Embed**	few, bounded, always read with the user
2	**Reference**	unbounded; optionally embed the 10 newest (*subset pattern*)
3	**Reference**	many-to-many; each side is read on its own
4	**Embed**	belongs to the order; a snapshot that never changes
5	**Both**	store `author_id` plus a copy of the name (*extended reference*)

★ **Key idea**

Ask three questions: Is it bounded? Is it read together with the parent? Does it change independently or belong to many parents?

08

Querying Data

Filters, projections, operators, sorting and pagination

- `find()` with filters and projections
- Comparison, logical, array and regex queries
- Counting and distinct values in PyMongo 4
- Sorting, `skip`/`limit` and range-based pagination

Loading the Sample Data

TERMINAL

```
$ mongoimport --db=world --collection=countries1 \
      --file=countries.json --jsonArray --drop
```

import_countries.py

```python
import json
from pathlib import Path

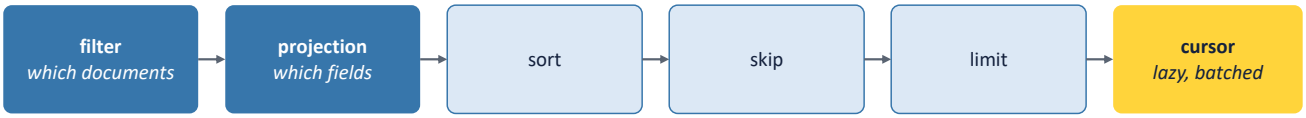
from pymongo import MongoClient

countries = json.loads(
    Path("countries.json").read_text(encoding="utf-8"))
with MongoClient() as client:
    coll = client["world"]["countries1"]
    coll.drop()  # start from a clean collection
    ids = coll.insert_many(countries).inserted_ids
    print(len(ids), "countries imported")
```

one document in world.countries1

```
{
  "_id": "TUR",
  "name": "Turkey",
  "continent": "Asia",
  "population": 66591000,
  "cities": [
    {"name": "Istanbul", "population": 8787958},
    {"name": "Ankara", "population": 3038159}
  ]
}
```


Anatomy of a find() Query



mongosh

```
db.customers.find()
db.customers.find({ city: "New York" })
db.customers.find(
  { city: "New York" },
  { name: 1, _id: 0 }
)
db.customers.findOne({ _id: 101 })
```

PyMongo

```
customers.find()
customers.find({"city": "New York"})
customers.find(
    {"city": "New York"},
    {"name": 1, "_id": 0},
)
customers.find_one({"_id": 101})
```

Key idea

`find()` returns a lazy **cursor**: nothing is fetched until you iterate, and results arrive in batches. `find_one()` returns a `dict` or `None`.

Comparison and Logical Operators

Operator	Matches when the field …
`$eq`, `$ne`	equals / differs from a value
`$gt`, `$gte`	is greater (or equal)
`$lt`, `$lte`	is less (or equal)
`$in`, `$nin`	is (not) one of a list
`$and`, `$or`, `$nor`	combines whole conditions
`$not`	negates an operator expression
`$exists`, `$type`	exists / has a BSON type

PyMongo

```
# price greater than 20
products.find({"price": {"$gt": 20}})

# 50 million < population < 100 million
countries.find(
    {"population": {"$gt": 50_000_000,
                    "$lt": 100_000_000}},
    {"name": True},
)

# in Europe OR more than 100 million people
countries.find({"$or": [
    {"continent": "Europe"},
    {"population": {"$gt": 100_000_000}},
]})
```


Counting and Distinct Values

mongosh

```
> db.countries1.countDocuments(
    { population: { $gt: 50000000 } })
24
> db.countries1.distinct("continent")
[ 'North America', 'Asia', 'Africa',
  'Europe', 'Oceania', 'South America' ]
```

PyMongo

```
big = {"population": {"$gt": 50_000_000}}
print(countries.count_documents(big))

continents = countries.distinct("continent")
print(len(continents), continents[:3])
```

SAMPLE OUTPUT

```
24
6 ['North America', 'Asia', 'Africa']
```

Watch out

`cursor.count()` and `Collection.count()` were **removed in PyMongo 4**. Use `count_documents(filter)` (exact) or `estimated_document_count()` (fast, from metadata).

Querying Dates

mongosh

```
db.orders.find({
  order_date: {
    $gte: ISODate("2023-01-01"),
    $lt: ISODate("2023-04-01")
  }
})
```

PyMongo

```
from datetime import datetime, timezone

UTC = timezone.utc
first_quarter = {
    "order_date": {
        "$gte": datetime(2023, 1, 1, tzinfo=UTC),
        "$lt": datetime(2023, 4, 1, tzinfo=UTC),
    }
}
for order in orders.find(first_quarter):
    print(order["_id"], order["order_date"])
```

Tip

Use a **half-open** range [start, end): `$lt` April 1 includes every instant of March 31, while `$lte` March 31 stops at 00:00 that day.

Querying Arrays and Embedded Documents

PyMongo

```
cursor = countries.find(
    {"cities.name": {"$in": ["Istanbul",
                             "New York"]}},
    {"name": True},
)
for country in cursor:
    print(country)
```

OUTPUT (WORLD DATABASE)

```
{'_id': 'TUR', 'name': 'Turkey'}
{'_id': 'USA', 'name': 'United States'}
```

Filter	Matches when …
`{"cities.name": "Ankara"}`	some city is Ankara
`{"tags": "vip"}`	the array contains `"vip"`
`{"tags": {"$all": ["a", "b"]}}`	it contains both
`{"tags": {"$size": 2}}`	it has 2 elements
`{"cities": {"$elemMatch": …}}`	one element meets all conditions

QUIZ

Quick Check: Counting

Which call counts the documents of collection `coll` that match the filter `q` in PyMongo 4?

A `coll.find(q).count()`

B `coll.count_documents(q)`

C `coll.estimated_document_count(q)`

D `len(coll.find(q))`

ANSWER

Quick Check: Counting

Which call counts the documents of collection `coll` that match the filter `q` in PyMongo 4?

A `coll.find(q).count()`

B `coll.count_documents(q)`

C `coll.estimated_document_count(q)`

D `len(coll.find(q))`

★ **Why**

`Cursor.count()` was removed in PyMongo 4, `estimated_document_count()` takes no filter, and a cursor has no `len()`. `count_documents(filter)` counts exactly on the server.

Regular Expressions

mongosh

```
db.countries1.find({ name: /^....$/ }, { name: 1, _id: 0 })
```

PyMongo

```
import re

four_letters = re.compile(r"^.{4}$")
cursor = countries.find({"name": four_letters},
                        {"name": 1, "_id": 0})
print(", ".join(doc["name"] for doc in cursor))

# the operator form: case-insensitive prefix
countries.find({"name": {"$regex": "^tur", "$options": "i"}})
```

OUTPUT (WORLD DATABASE)

```
Cuba, Guam, Iran, Laos, Mali, Niue, Oman, Peru, Chad
```

> **Watch out**
>
> Only a case-sensitive **prefix** pattern such as `^Tur` can use an index efficiently; `/tur/i` has to examine every key.

Sorting and Limiting Results

PyMongo

```
from pymongo import DESCENDING

top_asia = (
    countries.find({"continent": "Asia"},
                   {"name": 1, "population": 1, "_id": 0})
    .sort("population", DESCENDING)
    .limit(10)
)
for country in top_asia:
    print(f"{country['name']:<12}{country['population']:>14,}")
```

OUTPUT (WORLD DATABASE)

```
China        1,277,558,000
India        1,013,662,000
Indonesia      212,107,000
Pakistan       156,483,000
Bangladesh     129,155,000
Japan          126,714,000
Vietnam         79,832,000
Philippines     75,967,000
Iran            67,702,000
Turkey          66,591,000
```

mongosh

```
db.countries1.find({ continent: "Asia" }, { name: 1, population: 1, _id: 0 })
  .sort({ population: -1 }).limit(10)
```

Pagination with skip() and limit()

PyMongo

```python
def asia_page(page: int, size: int = 10) -> list[dict]:
    """Return page `page` (1-based) of Asia by population."""
    return list(
        countries.find({"continent": "Asia"},
                       {"name": 1, "population": 1, "_id": 0})
        .sort("population", DESCENDING)
        .skip((page - 1) * size)
        .limit(size)
    )


for country in asia_page(2)[:5]:
    print(country["name"], country["population"])
```

OUTPUT (WORLD DATABASE)

```
Thailand 61399000
South Korea 46844000
Myanmar 45611000
Uzbekistan 24318000
North Korea 24039000
```

Watch out

`skip(n)` still walks past *n* documents on the server: deep pages get slower, and inserts between requests shift items from one page to the next.

Range-Based Pagination

PyMongo

```python
def next_page(after: dict | None = None, size: int = 10) -> list[dict]:
    """Keyset pagination: return the page that follows `after`."""
    query: dict = {"continent": "Asia"}
    if after is not None:
        query["$or"] = [
            {"population": {"$lt": after["population"]}},
            {"population": after["population"],
             "_id": {"$gt": after["_id"]}},
        ]
    order = [("population", DESCENDING), ("_id", ASCENDING)]
    return list(countries.find(query).sort(order).limit(size))


first = next_page()
second = next_page(after=first[-1])
```

Key idea

Each page **seeks** to the last key seen instead of skipping: constant cost with an index on `(population, _id)` and stable pages — but you move page by page instead of jumping.

TRY IT

Query the World Database

Using `countries = client["world"]["countries1"]`, write PyMongo code that:

1. counts the European countries with more than 10 million people
2. lists only their `name` and `population`, largest first
3. shows the **second page**, five countries per page

> ✓ **Tip**
> Hint: reuse one filter dictionary for `count_documents` and `find`.

ANSWER

Query the World Database

```python
from pymongo import DESCENDING

query = {"continent": "Europe",
         "population": {"$gt": 10_000_000}}
print(countries.count_documents(query))

page_two = (
    countries.find(query, {"_id": 0, "name": 1, "population": 1})
    .sort("population", DESCENDING)
    .skip(5)
    .limit(5)
)
for country in page_two:
    print(country)
```

09

Aggregation Pipelines

Server-side data processing, stage by stage

- Pipelines, stages and their SQL counterparts
- Expressions, accumulators and `aggregate()` in PyMongo
- Hands-on: from `$group` to `$lookup`, `$facet` and `$bucket`

The Aggregation Framework

Process data **on the server** and send only the result to the application.

Reshape
custom presentations of data, e.g. read-only **views**

Join
combine collections with `$lookup` and `$unionWith`

Analyse
data wrangling, statistics, bucketing, facets

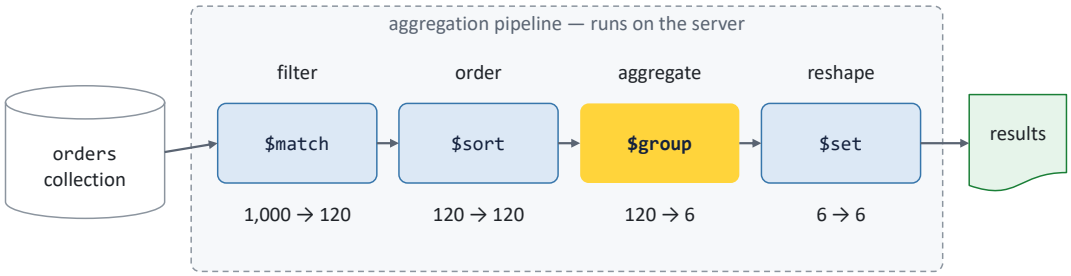
Scale
runs next to the data, also across a sharded cluster

Monitor
real-time analytics and dashboards

Any driver
the same pipeline from mongosh, Python, Java, …

A language of its own
More than 150 operators and expressions; formally the pipeline language is even Turing-complete.

Documents Flow Through Stages



PyMongo

```
pipeline = [
    {"$match": {"status": "shipped"}},
    {"$sort": {"order_date": 1}},
    {"$group": {"_id": "$customer_id",
                "first_order": {"$first": "$order_date"}}},
    {"$set": {"customer_id": "$_id"}},
]
results = db.orders.aggregate(pipeline)
```


SQL Terms and Aggregation Operators

SQL	Aggregation
`WHERE`	`$match`
`GROUP BY`	`$group`
`HAVING`	`$match` after `$group`
`SELECT`	`$project`, `$set`
`ORDER BY`	`$sort`
`LIMIT`	`$limit`
`OFFSET`	`$skip`

SQL	Aggregation
`SUM()`	`$sum`
`COUNT()`	`$sum: 1`, `$count`, `$sortByCount`
`JOIN`	`$lookup`
`SELECT INTO new_table`	`$out`
`MERGE INTO table`	`$merge` (MongoDB 4.2+)
`UNION ALL`	`$unionWith` (MongoDB 4.4+)

Key idea

SQL clauses have a fixed order; pipeline stages run **in the order you write them** and may repeat.

Expressions and Accumulators

Expression	Meaning
`"$price"`	value of the field `price`
`"$items.price"`	field in an embedded document
`{"$multiply": ["$a", "$b"]}`	operator expression
`{"$round": ["$avg", 2]}`	round to two decimals
`"$$NOW"`, `"$$item"`	system / user variables

Accumulator (in `$group`)	Result per group
`$sum`	total; `{"$sum": 1}` counts
`$avg`	average
`$min`, `$max`	smallest / largest value
`$first`, `$last`	value from the first / last document
`$push`, `$addToSet`	array of all / distinct values

Tip

A string starting with `$` is a field path; a key starting with `$` is an operator.

Aggregation in PyMongo

continents.py

```python
from pymongo import MongoClient

pipeline = [
    {"$match": {"population": {"$gt": 0}}},
    {"$group": {"_id": "$continent",
                "countries": {"$sum": 1},
                "people": {"$sum": "$population"}}},
    {"$sort": {"people": -1}},
]

with MongoClient("mongodb://localhost:27017") as client:
    countries = client["world"]["countries1"]
    for row in countries.aggregate(pipeline):
        print(row["_id"], row["countries"], row["people"])
```

Key idea

A pipeline is a plain `list` of `dicts`, so Python can build it with variables, loops and functions. `aggregate()` returns a `CommandCursor`.

TRY IT

Total Quantity per Product

Each document in `orders` is one order line. Compute the **total quantity** ordered for each product and list the largest totals first.

sample data

```
db.orders.insert_many([
    {"_id": 1, "product": "Pen", "quantity": 10},
    {"_id": 2, "product": "Notebook", "quantity": 5},
    {"_id": 3, "product": "Pen", "quantity": 3},
    {"_id": 4, "product": "Notebook", "quantity": 8},
])
```

Tip

Hint: `$group` by `"$product"`, accumulate with `$sum`, then `$sort`.

ANSWER

Total Quantity per Product

PyMongo

```
pipeline = [
    {"$group": {
        "_id": "$product",
        "total_quantity": {"$sum": "$quantity"},
    }},
    {"$sort": {"total_quantity": -1, "_id": 1}},
]
for row in db.orders.aggregate(pipeline):
    print(row)
```

RESULT

```
{'_id': 'Notebook', 'total_quantity': 13}
{'_id': 'Pen', 'total_quantity': 13}
```

mongosh

```
db.orders.aggregate([
  { $group: {
      _id: "$product",
      total_quantity: { $sum: "$quantity" }
  } },
  { $sort: { total_quantity: -1, _id: 1 } }
])
```

TRY IT

Filter Before Grouping

From `sales`, compute the **total sales per product** for products sold in **2023** only, largest first.

sample data

```
db.sales.insert_many([
    {"_id": 1, "product": "Laptop", "amount": 1000, "year": 2022},
    {"_id": 2, "product": "Phone", "amount": 600, "year": 2023},
    {"_id": 3, "product": "Laptop", "amount": 1100, "year": 2023},
    {"_id": 4, "product": "Tablet", "amount": 300, "year": 2023},
])
```

Tip

Hint: the SQL `WHERE` clause becomes a `$match` stage.

ANSWER

Filter Before Grouping

PyMongo

```
pipeline = [
    {"$match": {"year": 2023}},
    {"$group": {"_id": "$product",
                "total_sales": {"$sum": "$amount"}}},
    {"$sort": {"total_sales": -1}},
]
for row in db.sales.aggregate(pipeline):
    print(row)
```

RESULT

```
{'_id': 'Laptop', 'total_sales': 1100}
{'_id': 'Phone', 'total_sales': 600}
{'_id': 'Tablet', 'total_sales': 300}
```

Key idea

`$match` first: fewer documents reach `$group`, and the filter can use an index on `year`.

TRY IT

Computed Fields with $project

For each employee, output the `name` and a new field `total_salary` = `base_salary` × `months_worked`.

sample data

```
db.employees.insert_many([
    {"_id": 1, "name": "Alice", "base_salary": 3000, "months_worked": 12},
    {"_id": 2, "name": "Bob", "base_salary": 2500, "months_worked": 10},
    {"_id": 3, "name": "Charlie", "base_salary": 3500, "months_worked": 12},
])
```

Tip

Hint: inside `$project`, a field can be `1` (keep it) or an expression such as `$multiply`.

ANSWER

Computed Fields with $project

PyMongo

```
pipeline = [
    {"$project": {
        "name": 1,
        "total_salary": {"$multiply": ["$base_salary", "$months_worked"]},
    }},
]
for row in db.employees.aggregate(pipeline):
    print(row)
```

RESULT

```
{'_id': 1, 'name': 'Alice', 'total_salary': 36000}
{'_id': 2, 'name': 'Bob', 'total_salary': 25000}
{'_id': 3, 'name': 'Charlie', 'total_salary': 42000}
```

Key idea

`$project` keeps only the listed fields (plus `_id`); `$set` adds a field and keeps all the others.

TRY IT

One Document per Array Element

List each student together with each subject they take — one result document per (student, subject) pair.

sample data

```
db.students.insert_many([
    {"_id": 1, "name": "John", "subjects": ["Math", "Physics"]},
    {"_id": 2, "name": "Jane", "subjects": ["Chemistry", "Biology"]},
])
```

Tip

Hint: `$unwind` deconstructs an array field; `$project` can rename `subjects` to `subject`.

ANSWER

One Document per Array Element

PyMongo

```
pipeline = [
    {"$unwind": "$subjects"},
    {"$project": {"_id": 0, "name": 1, "subject": "$subjects"}},
]
for row in db.students.aggregate(pipeline):
    print(row)
```

RESULT

```
{'name': 'John', 'subject': 'Math'}
{'name': 'John', 'subject': 'Physics'}
{'name': 'Jane', 'subject': 'Chemistry'}
{'name': 'Jane', 'subject': 'Biology'}
```

Key idea

`$unwind` turns one document with *n* array elements into *n* documents.

TRY IT

Join Orders with Customers

Join `orders` with `customers` so that each order shows the product, the quantity and the **customer's name**.

orders

```
db.orders.insert_many([
    {"_id": 1, "customer_id": 101,
     "product": "Laptop", "quantity": 2},
    {"_id": 2, "customer_id": 102,
     "product": "Phone", "quantity": 1},
])
```

customers

```
db.customers.insert_many([
    {"_id": 101, "name": "John Doe"},
    {"_id": 102, "name": "Jane Smith"},
])
```

Tip

Hint: `$lookup` needs `from`, `localField`, `foreignField` and `as`.

ANSWER

Join Orders with Customers

PyMongo

```
pipeline = [
    {"$lookup": {
        "from": "customers",
        "localField": "customer_id",
        "foreignField": "_id",
        "as": "customer",
    }},
    {"$unwind": "$customer"},
    {"$project": {"_id": 0, "product": 1, "quantity": 1,
                  "customer": "$customer.name"}},
]
for row in db.orders.aggregate(pipeline):
    print(row)
```

RESULT

```
{'product': 'Laptop', 'quantity': 2, 'customer': 'John Doe'}
{'product': 'Phone', 'quantity': 1, 'customer': 'Jane Smith'}
```

TRY IT

Several Aggregations at Once

Using the `orders` data from the first exercise, compute in **one** pipeline both the number of documents and the average quantity.

orders

```
{"_id": 1, "product": "Pen", "quantity": 10}
{"_id": 2, "product": "Notebook", "quantity": 5}
{"_id": 3, "product": "Pen", "quantity": 3}
{"_id": 4, "product": "Notebook", "quantity": 8}
```

Tip

Hint: `$facet` runs several sub-pipelines on the same input documents; each one produces an array.

ANSWER

Several Aggregations at Once

PyMongo

```
pipeline = [
    {"$facet": {
        "total_documents": [{"$count": "count"}],
        "average_quantity": [
            {"$group": {"_id": None,
                        "avg_quantity": {"$avg": "$quantity"}}},
        ],
    }},
]
result = next(db.orders.aggregate(pipeline))
print(result["total_documents"][0]["count"])
print(result["average_quantity"][0]["avg_quantity"])
```

RESULT

```
4
6.5
```

Key idea

`$facet` returns **one** document with one array per sub-pipeline.

TRY IT

Bucketing Exam Scores

Group students into score ranges **0–50** (low), **51–80** (medium) and **81–100** (high); report how many students fall into each range and their names.

sample data

```
db.exam_scores.insert_many([
    {"_id": 1, "name": "Alice", "score": 45},
    {"_id": 2, "name": "Bob", "score": 78},
    {"_id": 3, "name": "Charlie", "score": 89},
])
```

Tip

Hint: `$bucket` uses `groupBy`, `boundaries` (lower bound inclusive, upper bound exclusive) and `output`.

ANSWER

Bucketing Exam Scores

PyMongo

```
pipeline = [
    {"$bucket": {
        "groupBy": "$score",
        "boundaries": [0, 51, 81, 101],
        "default": "unknown",
        "output": {
            "count": {"$sum": 1},
            "students": {"$push": "$name"},
        },
    }},
]
for row in db.exam_scores.aggregate(pipeline):
    print(row)
```

RESULT

```
{'_id': 0, 'count': 1, 'students': ['Alice']}
{'_id': 51, 'count': 1, 'students': ['Bob']}
{'_id': 81, 'count': 1, 'students': ['Charlie']}
```

The Shop Data Set

shop.orders

```
db.orders.insert_one({
    "_id": 1001,
    "customer_id": "C07",
    "salesperson": "Deniz",
    "order_date": datetime(2023, 4, 12,
                           tzinfo=timezone.utc),
    "items": [
        {"product": "Laptop",
         "category": "Electronics",
         "price": 900, "quantity": 1},
        {"product": "Mouse",
         "category": "Electronics",
         "price": 25, "quantity": 2},
    ],
})
```

shop.products

```
db.products.insert_one({
    "name": "Mouse",
    "category": "Electronics",
    "price": 25,
})
```

- One order → several embedded `items` with the **price paid**
- `customer_id` references a customer
- Revenue of an item = `price` × `quantity`

TRY IT

Top Five Customers by Spending

Find the **five customers who spent the most** in total over all their orders.

RESULT SHAPE (ILLUSTRATIVE VALUES)

```
{'_id': 'C07', 'total_spent': 4180}
{'_id': 'C02', 'total_spent': 3905}
...
```

Tip

Hint: `$unwind` the items, `$group` by customer summing price × quantity, then `$sort` and `$limit`.

ANSWER

Top Five Customers by Spending

PyMongo

```
pipeline = [
    {"$unwind": "$items"},
    {"$group": {
        "_id": "$customer_id",
        "total_spent": {"$sum": {
            "$multiply": ["$items.price", "$items.quantity"],
        }},
    }},
    {"$sort": {"total_spent": -1}},
    {"$limit": 5},
]
for row in db.orders.aggregate(pipeline):
    print(row["_id"], row["total_spent"])
```

Tip

`$sort` directly followed by `$limit` lets the server keep only the current top five in memory.

TRY IT

Best-Selling Categories

Find (a) the **most popular category** by total quantity sold, and (b) the **top three categories** by revenue. Try to share one pipeline builder for both.

RESULT SHAPE (ILLUSTRATIVE VALUES)

```
(a) {'_id': 'Stationery', 'total': 5230}
(b) {'_id': 'Electronics', 'total': 184250}
    {'_id': 'Books', 'total': 23875}
    ...
```

Tip

Hint: a Python function can return a pipeline; pass it the expression to sum and *n*.

ANSWER

Best-Selling Categories

PyMongo

```
REVENUE = {"$multiply": ["$items.price", "$items.quantity"]}


def top_categories(measure: str | dict, n: int) -> list[dict]:
    """Pipeline: the n categories with the largest sum of measure."""
    return [
        {"$unwind": "$items"},
        {"$group": {"_id": "$items.category",
                    "total": {"$sum": measure}}},
        {"$sort": {"total": -1}},
        {"$limit": n},
    ]


by_quantity = db.orders.aggregate(top_categories("$items.quantity", 1))
by_revenue = db.orders.aggregate(top_categories(REVENUE, 3))
```


TRY IT

Average Price per Category

Using `products`, compute the **average price in each category**, rounded to two decimals, most expensive category first.

RESULT SHAPE (ILLUSTRATIVE VALUES)

```
Electronics    312.40
Books           18.95
Stationery       3.10
```

Tip

Hint: `$avg` inside `$group`, then `$round` in a `$set` stage.

ANSWER

Average Price per Category

PyMongo

```
pipeline = [
    {"$group": {"_id": "$category",
                "avg_price": {"$avg": "$price"}}},
    {"$set": {"avg_price": {"$round": ["$avg_price", 2]}}},
    {"$sort": {"avg_price": -1}},
]
for row in db.products.aggregate(pipeline):
    print(f"{row['_id']:<12} {row['avg_price']:>8.2f}")
```

Watch out

Round **after** averaging: rounding every price first would change the result.

TRY IT

Orders in a Time Window

(a) Find the customers who placed an order in the **last 30 days**. (b) Compute the **revenue per salesperson** in **April 2023**.

RESULT SHAPE (ILLUSTRATIVE VALUES)

```
(a) {'_id': 'C02'}  {'_id': 'C11'}  ...
(b) {'_id': 'Deniz', 'revenue': 48210}  ...
```

Tip

Hint: compute the boundaries in Python with `datetime` and `timedelta`; use a half-open range.

ANSWER

Orders in a Time Window

PyMongo

```
from datetime import datetime, timedelta, timezone

UTC = timezone.utc
since = datetime.now(UTC) - timedelta(days=30)
recent_customers = [
    {"$match": {"order_date": {"$gte": since}}},
    {"$group": {"_id": "$customer_id"}},
]

april_revenue = [
    {"$match": {"order_date": {
        "$gte": datetime(2023, 4, 1, tzinfo=UTC),
        "$lt": datetime(2023, 5, 1, tzinfo=UTC)}}},
    {"$unwind": "$items"},
    {"$group": {"_id": "$salesperson", "revenue": {"$sum": {
        "$multiply": ["$items.price", "$items.quantity"]}}}},
]
```


TRY IT

Big Spenders and Their Order Counts

For every customer who spent **more than 1000** in total, report the amount spent and the **number of orders** they placed.

RESULT SHAPE (ILLUSTRATIVE VALUES)

```
{'_id': 'C07', 'total_spent': 4180, 'order_count': 6}
{'_id': 'C02', 'total_spent': 3905, 'order_count': 3}
...
```

Watch out

Careful: after `$unwind`, `{"$sum": 1}` counts **items**, not orders.

ANSWER

Big Spenders and Their Order Counts

PyMongo

```
ORDER_TOTAL = {"$sum": {"$map": {
    "input": "$items",
    "as": "item",
    "in": {"$multiply": ["$$item.price", "$$item.quantity"]},
}}}

pipeline = [
    {"$group": {
        "_id": "$customer_id",
        "total_spent": {"$sum": ORDER_TOTAL},
        "order_count": {"$sum": 1},
    }},
    {"$match": {"total_spent": {"$gt": 1000}}},
    {"$sort": {"total_spent": -1}},
]
```


9 · AGGREGATION PIPELINES

Writing Efficient Pipelines

Filter early

Start with `$match` (and `$sort`): only the first stages can use an **index**

Shrink early

`$project`/`$unset` unneeded fields before expensive stages

Top-k

`$sort` directly followed by `$limit` keeps only *k* documents in memory

Memory

100 MB per stage; larger sorts and groups **spill to disk** (default since 6.0)

Join late

`$lookup` after `$match`/`$limit`, with an index on `foreignField`

Persist

`$out` / `$merge` store results as a collection for dashboards

Tip

Check a pipeline's plan with `db.command("explain", ...)` or `.explain()` in mongosh — the Indexing section shows how to read it.

10

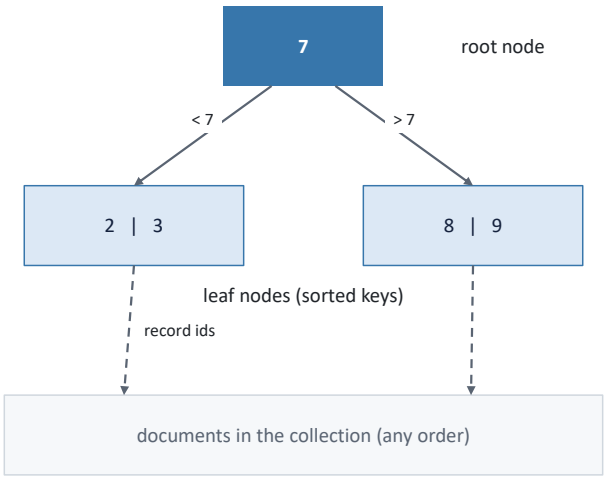
Indexing

From collection scans to index scans

- B-tree indexes and what they speed up
- `explain()`: `COLLSCAN` vs `IXSCAN`
- Compound, unique, partial and TTL indexes
- Covered queries and the cost of indexes

What Is an Index?

- A special data structure for **fast access** to the documents of a collection
- MongoDB uses **B-trees**: balanced, sorted, O(log n) search, insert and delete
- Each key points to its document, so a search compares few keys
- Every collection has one on `_id`; you add others with `create_index`



Without an Index: COLLSCAN

mongosh

```
db.movies1.find({ year: 2010 })
  .explain("executionStats")
```

PyMongo

```
plan = movies.find({"year": 2010}).explain()
stats = plan["executionStats"]
print(stats["totalDocsExamined"],
      stats["nReturned"])
```

OUTPUT (IMDB DATABASE)

```
250 18
```

Watch out

Every document was read to return 18: **250 examined / 18 returned**.

EXPLAIN OUTPUT (EXCERPT)

```
executionStats: {
  executionSuccess: true,
  nReturned: 18,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 250,
  executionStages: {
    stage: 'COLLSCAN',
    filter: { year: { '$eq': 2010 } },
    nReturned: 18,
    works: 251,
    direction: 'forward',
    docsExamined: 250
  }
}
```


With an Index: IXSCAN

mongosh

```
db.movies1.createIndex({ year: 1 })
db.movies1.find({ year: 2010 })
  .explain("executionStats")
```

PyMongo

```
print(movies.create_index("year"))
plan = movies.find({"year": 2010}).explain()
stats = plan["executionStats"]
print(stats["totalKeysExamined"],
      stats["totalDocsExamined"])
```

OUTPUT (IMDB DATABASE)

```
year_1
18 18
```

EXPLAIN OUTPUT (EXCERPT)

```
executionStats: {
  nReturned: 18,
  totalKeysExamined: 18,
  totalDocsExamined: 18,
  executionStages: {
    stage: 'FETCH',
    nReturned: 18,
    docsExamined: 18,
    inputStage: {
      stage: 'IXSCAN',
      keyPattern: { year: 1 },
      indexName: 'year_1',
      nReturned: 18
    }
  }
}
```

Compound Indexes and Key Order

PyMongo

```python
from pymongo import ASCENDING, DESCENDING

countries.create_index(
    [("continent", ASCENDING),
     ("population", DESCENDING)])

# can use the index: continent is a prefix
countries.find({"continent": "Asia"})
countries.find({"continent": "Asia"}).sort(
    "population", DESCENDING)

# cannot seek: population alone is no prefix
countries.find(
    {"population": {"$gt": 50_000_000}})
```

continent ↑	population ↓
Africa	111,506,000
Africa	68,470,000
…	…
Asia	1,277,558,000
Asia	1,013,662,000
…	…
Europe	146,934,000

Tip

Order keys by **E**quality, **S**ort, **R**ange (the ESR guideline).

Unique, Partial and TTL Indexes

PyMongo

```python
users.create_index("email", unique=True)

users.create_index(
    "phone",
    unique=True,
    partialFilterExpression={"phone": {"$exists": True}},
)

sessions.create_index("created_at", expireAfterSeconds=3600)
```

Option	Effect	Typical use
`unique=True`	rejects duplicate keys with `DuplicateKeyError`	e-mail, user name
`partialFilterExpression`	indexes only documents matching a filter	optional fields
`sparse=True`	skips documents without the field	older form of the above
`expireAfterSeconds`	a background task deletes expired documents	sessions, caches, logs

Covered Queries

PyMongo

```
cursor = countries.find(
    {"continent": "Asia"},
    {"_id": 0, "continent": 1, "population": 1},
)
stats = cursor.explain()["executionStats"]
print(stats["totalKeysExamined"],
      stats["totalDocsExamined"])
```

SAMPLE OUTPUT

```
51 0
```

- Every field in the filter **and** in the projection is part of one index
- `_id` is excluded unless it is in the index
- The plan shows `PROJECTION_COVERED` over `IXSCAN` and **0 documents examined**

Key idea

Uses the compound index `(continent, population)` from two slides ago: the answer comes straight from the B-tree.

What Indexes Cost

mongosh

```
> db.cities.storageSize()
696320
> db.cities.totalIndexSize()
122640
> db.cities.totalSize()
818960
```

PyMongo

```
stats = next(cities.aggregate(
    [{"$collStats": {"storageStats": {}}}]
))["storageStats"]
print(stats["count"], stats["totalIndexSize"])
print(cities.index_information())
```

SAMPLE OUTPUT

```
4078 122640
{'_id_': {'v': 2, 'key': [('_id', 1)]}}
```

- Disk and RAM: each index is a B-tree that should fit in memory
- Writes: every insert, update and delete updates **every** index
- Keep indexes that serve real queries; find unused ones with `$indexStats`
- Inspect with `index_information()`, remove with `drop_index("year_1")`

QUIZ

Quick Check: Compound Indexes

An index exists on `(continent, population)`. Which query can **not** seek into it?

A `find({"continent": "Asia"})`

B `find({"continent": "Asia"}).sort("population", -1)`

C `find({"population": {"$gt": 100_000_000}})`

D `find({"continent": "Europe", "population": {"$lt": 1_000_000}})`

ANSWER

Quick Check: Compound Indexes

An index exists on `(continent, population)`. Which query can **not** seek into it?

A `find({"continent": "Asia"})`

B `find({"continent": "Asia"}).sort("population", -1)`

C `find({"population": {"$gt": 100_000_000}})`

D `find({"continent": "Europe", "population": {"$lt": 1_000_000}})`

★ **Why**

Only queries that constrain a **prefix** of the key (`continent`) can jump into the index. Population alone is spread over every continent, so the server would scan the whole index or collection.

TRY IT

Summarise an explain() Plan

`explain()` returns nested dictionaries. Write `plan_summary(plan)` that follows `inputStage` down to the **leaf stage** and reports its name, the index used and the examined counts.

two plans (trimmed)

```
IXSCAN_PLAN = {"executionStats": {
    "nReturned": 18, "totalKeysExamined": 18,
    "totalDocsExamined": 18,
    "executionStages": {
        "stage": "FETCH",
        "inputStage": {"stage": "IXSCAN",
                       "indexName": "year_1"}}}}
COLLSCAN_PLAN = {"executionStats": {
    "nReturned": 18, "totalKeysExamined": 0,
    "totalDocsExamined": 250,
    "executionStages": {"stage": "COLLSCAN"}}}
```

```
>>> plan_summary(IXSCAN_PLAN)
'IXSCAN year_1: keys=18 docs=18 returned=18'
>>> plan_summary(COLLSCAN_PLAN)
'COLLSCAN -: keys=0 docs=250 returned=18'
```


ANSWER

Summarise an explain() Plan

```python
def plan_summary(plan: dict) -> str:
    """Summarise the winning plan of an explain() result."""
    stats = plan["executionStats"]
    stage = stats["executionStages"]
    while "inputStage" in stage:
        stage = stage["inputStage"]
    index = stage.get("indexName", "-")
    return (f"{stage['stage']} {index}: "
            f"keys={stats['totalKeysExamined']} "
            f"docs={stats['totalDocsExamined']} "
            f"returned={stats['nReturned']}")


print(plan_summary(IXSCAN_PLAN))
print(plan_summary(COLLSCAN_PLAN))
```

OUTPUT

```
IXSCAN year_1: keys=18 docs=18 returned=18
COLLSCAN -: keys=0 docs=250 returned=18
```

11

Async PyMongo

AsyncMongoClient with asyncio and FastAPI

- Why blocking database calls hurt async programs
- `AsyncMongoClient`: `await`, `async for`, `async with`
- Concurrent queries and a FastAPI preview

Why an Async Driver?

- An async server runs many requests on **one event loop** (Module 5)
- A blocking `MongoClient` call freezes the loop while it waits for the database
- `AsyncMongoClient` makes every database round trip **awaitable**

Synchronous (`MongoClient`)	Asynchronous (`AsyncMongoClient`)
`with MongoClient() as client:`	`async with AsyncMongoClient() as client:`
`doc = coll.find_one(query)`	`doc = await coll.find_one(query)`
`for doc in coll.find(query):`	`async for doc in coll.find(query):`
`docs = list(coll.find(query))`	`docs = await coll.find(query).to_list()`
`coll.aggregate(pipeline)`	`await coll.aggregate(pipeline)`

Python version note

Async API: beta in PyMongo 4.10, generally available since 4.13; **Motor is deprecated** — new code uses `AsyncMongoClient`.

A First AsyncMongoClient Program

async_countries.py

```python
import asyncio

from pymongo import AsyncMongoClient


async def main() -> None:
    async with AsyncMongoClient("mongodb://localhost:27017") as client:
        countries = client["world"]["countries1"]
        turkey = await countries.find_one({"_id": "TUR"})
        print(turkey["name"])
        print(await countries.count_documents({"continent": "Asia"}))
        europe = countries.find({"continent": "Europe"}).limit(3)
        async for country in europe:
            print(country["name"])


if __name__ == "__main__":
    asyncio.run(main())
```

SAMPLE OUTPUT

```
Turkey
51
Albania
Andorra
Austria
```

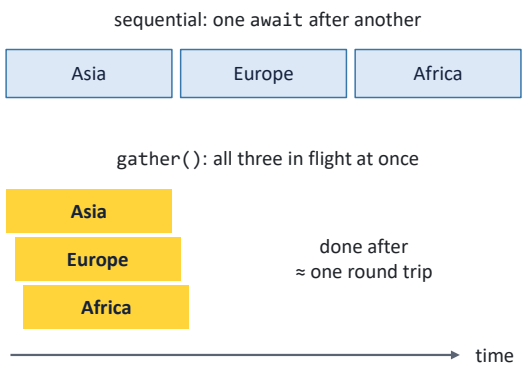

Running Queries Concurrently

PyMongo (async)

```python
import asyncio

from pymongo.asynchronous.collection import (
    AsyncCollection)


async def continent_sizes(
        countries: AsyncCollection,
        names: list[str]) -> dict[str, int]:
    """Count several continents concurrently."""
    counts = await asyncio.gather(*(
        countries.count_documents({"continent": name})
        for name in names))
    return dict(zip(names, counts))
```



Async PyMongo in a FastAPI Endpoint

app.py (setup)

```python
from contextlib import asynccontextmanager

from fastapi import FastAPI, HTTPException
from pymongo import AsyncMongoClient

MONGO_URL = "mongodb://localhost:27017"


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.mongo = AsyncMongoClient(MONGO_URL)
    yield
    await app.state.mongo.close()


app = FastAPI(lifespan=lifespan)
```

app.py (endpoint)

```python
@app.get("/countries/{code}")
async def get_country(code: str) -> dict:
    db = app.state.mongo["world"]
    country = await db.countries1.find_one(
        {"_id": code}, {"_id": 0})
    if country is None:
        raise HTTPException(404, "not found")
    return country
```

i While one request awaits MongoDB, the event loop serves the others. Module 8 builds complete REST APIs on this pattern.

QUIZ

Quick Check: Async PyMongo

In a coroutine using `AsyncMongoClient`, which line correctly reads one document from `coll`?

A `doc = coll.find_one(q)`

B `doc = await coll.find_one(q)`

C `doc = await coll.find(q)`

D `doc = list(coll.find(q))[0]`

ANSWER

Quick Check: Async PyMongo

In a coroutine using `AsyncMongoClient`, which line correctly reads one document from `coll`?

A `doc = coll.find_one(q)`

B `doc = await coll.find_one(q)`

C `doc = await coll.find(q)`

D `doc = list(coll.find(q))[0]`

Why

`find_one` is a coroutine and must be awaited. `find` only builds a cursor: iterate it with `async for` or collect it with `await cursor.to_list()`; `list()` does not accept it.

11 · ASYNC PYMONGO

Practice Problems

1 · Country explorer

A CLI (`argparse`) that filters `world.countries1` by continent and population, sorts, and pages with range-based pagination

2 · Validated students

Create `students` with `$jsonSchema`; insert valid and invalid documents and report each `WriteError`

3 · Library model

Model books, authors and loans (embed or reference?); list overdue loans with `$lookup`

4 · Sales dashboard

Monthly revenue with `$dateTrunc`, top categories and a price histogram in one `$facet`

5 · Index tuning

Insert 100,000 random orders; compare `explain()` before and after a compound index; build a covered query

6 · Async importer

Load several JSON files concurrently with `AsyncMongoClient`, `asyncio.gather` and `insert_many`

MODULE 07 · SUMMARY

Key Takeaways

- MongoDB stores **BSON documents** in collections; replica sets add availability, sharding adds scale
- PyMongo 4: `MongoClient` as context manager, `insert_*`, `find`, `update_*` with operators, `delete_*`, `count_documents`
- Know your BSON types (UTC dates, `Decimal128`, `ObjectId`) and enforce shapes with `$jsonSchema`
- Model for access patterns: **embed** what is read together, **reference** the rest, join with `$lookup`
- Query with operators, projections, `sort` and range-based pagination; compute with aggregation pipelines
- Indexes turn `COLLSCAN` into `IXSCAN` — verify with `explain()`
- `AsyncMongoClient` brings the same API to `asyncio` and FastAPI

NEXT UP Module 08 — Designing and Implementing RESTful APIs with Flask and FastAPI